

Lecture 11 — Technical Debt

Lecture id: L11 **Source decks:** BeyondTechnicalDebt (~26p) **Labs:** None **Process phase(s):** Cross-cutting (software evolution / maintenance economics) **Citation key:** (BeyondTechnicalDebt p.X); readings [Raj13] etc. **Grounding note:** Every non-obvious claim below is cited to a specific slide of the BeyondTechnicalDebt deck (26 pages, all read in full). The deck is image-heavy: most slides are screenshots (CodeScene dashboards, the "Actionable?" Sonar report, the fault-prediction paper, classical paintings used as metaphors) with little or no extractable text, so the page renders were read visually. Where the slide cites an external source verbatim (Ford/Parsons/Kua, Rich Hickey, Graves et al., Adam Tornhill's CodeScene), that attribution is preserved. Connections to the Rajlich change process, maintainability, and refactoring are drawn by this guide for exam purposes and are flagged as such; the deck itself does not name Rajlich's phases. Reading keys ([Raj13], [MC09], [Fowler99], [Martin]) refer to the course bibliography in 00-overview.md. The deck's own headline external references are Neal Ford, Rebecca Parsons & Patrick Kua, *Building Evolutionary Architectures* (p.110), and Adam Tornhill's *Your Code as a Crime Scene* body of work (the CodeScene / "crime scene" tooling at adamtornhill.com).

Overview

Lecture 11 is the course's "beyond" lecture: a cross-cutting capstone that steps back from the mechanics of a single change and asks how *accumulated* quality problems — **technical debt** — build up across the lifetime of a system, why conventional static-analysis tools fail to surface the debt that actually matters, and how *behavioural / evolutionary* code analysis (mining the version-control history) can prioritise debt by its real cost to the organisation. The framing device for the whole deck is the title slide's tortoise crawling down a long road (BeyondTechnicalDebt p.1): debt slows you down for the long haul.

The lecture opens by **defining technical debt** through a quotable formulation — "*Stuff that isn't supposed to be there and is in the way of the stuff that is supposed to be there*" (Ford, Parsons & Kua, *Building Evolutionary Architectures* p.110, cited on BeyondTechnicalDebt p.2). It then grounds debt in **Lehman's Laws of software evolution** (Continuing Change, Increasing Complexity; p.3) and in the economic argument that **complexity kills development speed** (Rich Hickey, *Simple Made Easy*; p.4) and has direct **business and product impact** — long lead times, lack of predictability, and bugs users experience (p.5).

The middle of the deck is the methodological turn. Slide 6 ("Conventional Tools — What is missing?") and slide 7 ("Actionable?") argue that traditional static analysers (e.g. a SonarQube-style dashboard reporting **Technical Debt: 11.0%, 683 man days, 10,072 violations**) give you a **snapshot** but no sense of *where to start* — "Thousands of years of technical debt" (the Tower of Babel, p.8): "Where do you start when you want to pay it back?" The answer the deck advocates is **CodeScene** (Adam Tornhill's tool): a "*movie rather than a snapshot*" (p.9) that adds two dimensions conventional tools miss — **the time aspect** (evolution over the VCS history) and **organization & people** (who works on the code, how, and how often) (p.10). Static analysis alone can never tell you whether excess complexity *actually matters*; CodeScene prioritises debt **based on how the organization works with the code** (p.10).

The core analytical concept is the **hotspot**: "*complicated code that you have to work with often*" (p.12) — the intersection of high **code complexity** (the *principal*) and high **code change frequency** (the *interest rate*) (p.12). Hotspots are visualised as nested-circle enclosure diagrams (ReactJS, p.13), correlate strongly with

bug incidence (Graves et al., *Predicting Fault Incidence Using Software Change History*, p.14), reveal **worrisome trends** (a TypeScript `checker.ts` with a rising complexity curve, p.15), and can be drilled into with **X-Ray** to find the most problematic *functions* (p.16–18) and **change coupling** between files (p.19).

The deck closes with the social half of debt — **legacy code** — arguing "*why technical debt isn't just technical*" (p.20): legacy code is defined by **quality** (relative) and **ownership** (code we didn't write ourselves), introducing **knowledge loss**, **off-boarding risk**, and the "*technical debt that wasn't*" — abandoned code nobody owns (p.21–23). The concluding messages (p.25): technical debt is a real, language-independent problem; the version-control system holds a huge amount of useful information; but ultimately you must **rely on human expertise**, and the right role of data is to **support developers' judgement and experience to get the highest ROI**.

For the change process this lecture is the *economics layer over* refactoring (LO4) and maintainability: it tells you **which** debt to repay and **when**, so that **Prefactoring** and **Postfactoring** effort is spent where it yields the most return rather than spread thinly across the whole codebase.

Learning Objectives

After this lecture you should be able to:

1. **Define technical debt** and articulate the financial metaphor — *principal* (the substandard code) vs *interest* (the extra cost paid every time you work near it) — using the deck's working definition: stuff that is in the way of the stuff that is supposed to be there (BeyondTechnicalDebt p.2, p.12).
2. **Connect debt to software evolution** via Lehman's Laws: a system must be *continually adapted* or it becomes progressively less satisfactory (Continuing Change), and its *complexity increases* unless work is done to maintain or reduce it (Increasing Complexity) (p.3).
3. **Explain why complexity is an economic problem**, not just an aesthetic one: ignored complexity slows every sprint and "eventually kills you" (Hickey, p.4), and propagates to **business** (long lead times, unpredictability) and **product** (bugs users experience) (p.5).
4. **Critique conventional static-analysis tools**: they produce a *snapshot* of violations and a single debt figure (e.g. 11.0% / 683 man days) but cannot tell you whether complexity *matters* or **where to start repaying** (p.6–8, p.10).
5. **Describe behavioural / evolutionary code analysis** (CodeScene): a *movie* not a snapshot, adding the **time aspect** and **organization & people** dimensions by mining **code, process and evolutionary metrics** from **source code + version-control data + project-management tools (JIRA)** through **pattern detectors / ML** (p.9–11).
6. **Define and identify a hotspot** as the product of **code complexity (principal) × change frequency (interest rate)**, read a nested-circle hotspot map, and explain why hotspots are the rational first target for repayment (p.12–13).
7. **Justify hotspots empirically**: change-history process measures predict faults better than product/size metrics; the number of times a module changed predicts its fault count better than its length (Graves et al., p.14).
8. **Interpret a hotspot deep-dive**: read a "worrisome trend" (rising complexity vs flat LOC, defect ratio, knowledge loss) (p.15) and use **X-Ray** to rank functions by change frequency × cyclomatic complexity, and to surface **change coupling** between files (p.16–19).
9. **Explain the social dimension of debt**: *legacy code* = code that lacks quality (relative) and/or that we didn't write ourselves; reason about **knowledge loss**, **off-boarding risk**, and **simulated off-boarding** to predict how quickly a codebase turns into legacy code (p.20–23).

10. **State the lecture's bottom line:** debt is real and language-independent; the VCS is rich in information; but tools support — they do not replace — **human expertise**, and the goal of data is highest-ROI decisions (p.25).
11. **Relate all of the above to the change process:** position debt management as the prioritisation layer that directs Prefactoring/Postfactoring (refactoring) effort, and explain how an impacted hotspot raises the risk and cost of a change.

Key Concepts

The technical-debt metaphor — definition

What it is. The deck's working definition (Ford, Parsons & Kua, *Building Evolutionary Architectures* p.110): technical debt is "*Stuff that isn't supposed to be there and is in the way of the stuff that is supposed to be there.*" (BeyondTechnicalDebt p.2). In plainer terms, technical debt is the gap between the code you have and the clean, well-designed code you *would* have written if there had been no time pressure, no missing knowledge, and no accumulated evolutionary cruft — the awkward workarounds, copy-pasted blocks, overgrown functions, dead branches, and tangled dependencies that have crept in. The emphasised clause — *and is in the way* — is the load-bearing part: clutter that nobody touches is harmless; debt only "costs" when it obstructs the work you actually need to do. This is why the deck later argues that *where* the debt sits (in code you change often) matters far more than *how much* of it there is.

The metaphor itself (originally Ward Cunningham's) frames substandard-but-shipped code as a financial loan: you borrow time now (ship faster by cutting a corner) and repay it later, with **interest**, every time you have to work around the mess. The deck operationalises the two halves of the loan explicitly on the hotspot slide (p.12): **Principal** = the code complexity you took on; **Interest Rate** = the frequency with which you have to work with that code. (See *principal vs interest* below.)

What it's used for / why it matters. The metaphor exists to make an *engineering* quality problem legible to *non-engineers* — managers, product owners, the business. Calling it "debt" communicates three things a manager already understands: (1) you can deliberately take it on to hit a deadline (a reasonable, even strategic, loan); (2) it does not stay constant — it accrues interest, so the longer you leave it the more it costs; and (3) at some point the interest payments crowd out everything else and you can no longer afford new features. It reframes "we should clean up the code" (which sounds like developer perfectionism) as "we are paying interest on a loan that is eating our delivery capacity" (which sounds like a business decision). That reframing is what lets a team *negotiate time* to repay debt rather than being told to keep shipping features.

When & how it's applied. Concretely: a team under deadline pressure ships an order-processing module by hard-coding three special-case discounts instead of building a proper rule engine. That shortcut is the *principal*. Six months later every new discount rule means editing the same tangled `if/else` block, re-reading it, re-testing it, and occasionally re-introducing a bug — that recurring extra effort is the *interest*. When the team finally builds the rule engine they intended, that is *repaying the principal*. The lecture's whole apparatus (hotspots, X-Ray, trends) exists to tell a team *which* of their many such loans is charging the most interest, so they repay that one first instead of trying to repay all of them (which the Tower-of-Babel slide, p.8, shows is hopeless).

Change-process link: Debt is the negative residue that **Prefactoring** (clean the landing site before a change) and **Postfactoring** (clean up after a change) exist to remove. The whole point of bracketing **Actualization** with two refactoring phases (`00-overview.md` ; [Raj13]) is to stop each change from

adding to the principal. This lecture supplies the *prioritisation* that decides where that refactoring effort is worth spending.

Lehman's Laws — why debt is inevitable

What they are. Lehman's "Laws" of software evolution are a set of empirical generalisations (Manny Lehman, from studies of large industrial systems like OS/360) describing how *real, in-use* software behaves as it ages. They are not laws in the physics sense — they are observed regularities about so-called **E-type systems** (software that solves a real-world problem and is embedded in the world it models). The deck cites two of them (BeyondTechnicalDebt p.3):

- **Continuing Change** — "*a system must be continually adapted or it becomes progressively less satisfactory.*" A useful system lives in a changing world (new regulations, new hardware, new user expectations); if you stop changing it, the world drifts away from it and its fitness decays even though the code itself never moved.
- **Increasing Complexity** — "*as a system evolves, its complexity increases unless work is done to maintain or reduce it.*" Complexity creep is the *default* trajectory: each change, made under local pressure, tends to add a special case, a new dependency, or an extra branch. Keeping complexity flat requires deliberate, continuous, *separately budgeted* effort — it does not happen on its own.

What they're used for / why they matter. These two laws are the deck's *theoretical proof that debt is structural, not a sign of a bad team*. Put together, they form a trap: Law 1 says you are *required* to keep changing the system (you cannot freeze it without it rotting), and Law 2 says every change *tends* to raise complexity. So debt accumulates unless you actively spend effort countering it — even a disciplined team accrues debt simply by being forced to evolve a living system. Used in an exam or a planning discussion, this justifies treating refactoring not as a one-off "cleanup sprint" but as a **permanent, recurring tax** on development, and it motivates the rest of the deck: if debt is unavoidable, the interesting question is no longer "how do we avoid it?" but "how do we *prioritise* paying it down?"

When & how it's applied. A manager asks "why does this codebase need refactoring time *every* quarter — didn't we clean it up last year?" The Lehman answer: because the product is still being changed (Continuing Change), and each of those changes added complexity (Increasing Complexity), the system is *necessarily* more tangled than it was last year; the cleanup is not undoing a one-time mistake but counteracting an ongoing physical tendency. Practically, teams operationalise this by reserving a standing fraction of capacity (e.g. 15–20% per sprint) for complexity reduction, and they use the *behavioural metrics* in the rest of this lecture to aim that standing budget at the files where complexity is rising fastest (see *worrisome trends*, p.15).

Why complexity kills development speed (the economic argument)

What it is. This concept is the *velocity* framing of debt. Slide 4 (BeyondTechnicalDebt p.4) quotes **Rich Hickey** (*Simple Made Easy*): "*if you ignore complexity, you will slow down. You will invariably slow down over the long haul ... the complexity will eventually kill you. It will kill you in a way that will make every sprint accomplish less.*" The accompanying chart plots **speed against time** for "Easy" vs "Simple" code: the easy/complex path starts fast (cutting corners feels productive at first) but its speed collapses as the accumulated tangle makes every subsequent change harder; the simple path is slower to start but *sustains* throughput because it stays comprehensible. Hickey's distinction is important: "*easy*" means convenient-right-now (familiar, close at hand, quick to bolt on), whereas "*simple*" means un-entangled (one concept, one responsibility) — and chasing easy at the expense of simple is exactly how complexity accrues.

What it's used for / why it matters. The point is to *quantify the cost of debt as lost velocity over time*, so that refactoring can be argued for as an investment rather than a luxury. Technical debt is not primarily an aesthetic or "purist" concern — it is a **velocity** and **economics** problem. Because unmanaged complexity *compounds* (each tangle makes the next change harder, which under pressure adds more tangle), each sprint delivers less than the last. This is the argument a developer uses to push back on "no time to refactor, just ship it": refusing to manage complexity does not buy speed, it *borrow*s speed from every future sprint at a compounding rate. It reframes refactoring spend as an *investment in future delivery speed*, not a cost.

When & how it's applied. A team notices that the same kind of feature that took 3 days a year ago now takes 2 weeks, and the difference is all "understanding the existing code and not breaking it." That is the easy-vs-simple curve made real: they are at the point where the complex path's speed has collapsed. The economic argument is then used to justify pausing feature work to invest in simplification of the hotspots driving that slowdown — and the rest of the lecture (hotspot prioritisation) tells them *exactly which* complexity is doing the most damage to velocity, so the investment is targeted rather than a blanket (and unaffordable) rewrite.

The Hickey quotation and the Easy-vs-Simple chart, in full

The verbatim slide text. Slide 4 (BeyondTechnicalDebt p.4) carries the Rich Hickey quotation in full, with the slide's own bold emphasis: *"It's my contention, based on experience, that if you ignore complexity, you will slow down. You will invariably slow down over the long haul ... the complexity will eventually kill you. It will kill you in a way that will make every sprint accomplish less. — Rich Hickey, Simple Made Easy."* Three details of the wording are worth memorising for the exam. First, *"based on experience"*: Hickey is making a practitioner's contention, not reporting a controlled study — the deck's *empirical* leg comes later, from Graves et al. (p.14), so if an exam question asks for the deck's *evidence* you cite Graves, and if it asks for the deck's *motivating argument* you cite Hickey. Second, *"invariably"* and *"over the long haul"*: the claim is about inevitability and time horizon — ignoring complexity does not *sometimes* slow you down, it *always* does, and the damage shows up late, which is exactly why teams keep getting away with it sprint after sprint until they don't. Third, *"make every sprint accomplish less"*: the unit of damage is the *sprint* — the quotation is deliberately phrased in agile-delivery vocabulary so the cost lands with people who plan sprints, not just people who read code (BeyondTechnicalDebt p.4).

The chart's anatomy. The right half of the slide is a small area chart with **Speed on the y-axis and Time on the x-axis**, and a two-entry legend: **Easy** (blue) and **Simple** (green) (BeyondTechnicalDebt p.4). The blue *Easy* area starts high at the origin — taking the convenient shortcut genuinely is faster at first — and then decays toward the x-axis as time passes. The green *Simple* area starts lower — doing the un-entangled design costs more up front — but rises and then *dominates* the chart for the rest of the timeline. The crossover point where green overtakes blue is the **break-even** of the simple investment: before it, the shortcut team looks better in every status meeting; after it, the simple team ships faster forever. The exam-ready reading: the chart is the technical-debt loan drawn as a velocity curve — the blue area's early advantage is the *borrowed* speed, and the gap between green and blue on the right side is the *interest* being paid back, sprint after sprint (BeyondTechnicalDebt p.4).

How to use it. When a question asks "why is technical debt an economic rather than aesthetic problem?", reproduce this chart in words: complexity ignored = early speed, late collapse; complexity managed = early cost, sustained throughput; and the longer the system lives, the more the simple path wins — which connects directly back to Lehman's Continuing Change (p.3): systems that matter are exactly the ones that live long enough for the curves to cross (BeyondTechnicalDebt p.3–4).

Business & product impact

What it is. Slide 5 (BeyondTechnicalDebt p.5) makes the abstract cost of debt concrete by showing its two-sided *external* impact — the symptoms a non-developer actually observes — with a "BILL" (invoice) figure in the middle paying "Technical Debt" out to both sides:

- **Roadmap / Business side** — *what the business sees*: **long lead times** (features take longer and longer to ship) and **lack of predictability** (estimates become unreliable because nobody can foresee what the tangled code will do). (The slide uses a tormented classical painting to dramatise the roadmap pain.)
- **Product side** — *what the users experience*: **bugs** (drawn as a swarm of beetles/bugs) — defects that escape into production because complex, frequently-changed code is hard to test thoroughly.

What it's used for / why it matters. This slide is the bridge from "internal code quality" to "things the organisation already cares about and measures." It is used to make the case that debt is a *business* liability, not merely a developer inconvenience: it manifests outward as slow, unpredictable delivery (felt by the business and its customers) and as defects (felt by users, eroding trust). That outward manifestation is *why debt is worth measuring and prioritising at all* — if debt had no business consequence there would be nothing to manage. It also sets up the deck's closing frame: because the cost is real money and real lost trust, the right lens is **highest ROI** (p.25) — spend repayment effort where it most reduces lead time, unpredictability, and bug rate.

When & how it's applied. When a developer needs to win a refactoring budget, this is the vocabulary to use in the planning meeting: rather than "the code is ugly," say "our lead time on this module has doubled and we can't give reliable dates (business impact), and it's generating a disproportionate share of our production bugs (product impact) — both trace back to debt in these specific files." Tying the abstract debt to lead-time, predictability, and bug metrics that the business already tracks is what converts a quality argument into a fundable one. The hotspot analysis later in the deck is then the tool that identifies *which* files are generating those business/product symptoms.

Slide 5's diagram anatomy — Roadmap, BILL, Product

What the slide actually shows. The impact slide (BeyondTechnicalDebt p.5, headline "*Technical Debt Has Impact on Business & Product*") is a three-part diagram worth being able to reproduce from memory, because each visual element maps to one named cost. In the **centre** stands a cartoon figure clutching an oversized invoice labelled "**BILL**" — the personification of the debt coming due. From this figure two arrows labelled "**Technical Debt**" radiate outward, one to each side, showing that the *same* bill is paid twice, once by each constituency. On the **left**, under the heading **Roadmap**, sits a tormented classical painting (a figure straining under a crushing burden — the Sisyphus image) annotated "*What the business see*", with the two named costs printed beneath it: **Long Lead Times** and **Lack of Predictability** (BeyondTechnicalDebt p.5). On the **right**, under the heading **Product**, a swarm of drawn beetles is annotated "*What the users experience*" — the **bugs** that escape into production (BeyondTechnicalDebt p.5).

Why the layout matters. The deliberate symmetry — one bill, two payees — is the slide's argument compressed into geometry: technical debt is not an internal engineering ledger entry; it is invoiced *outward*, simultaneously, to the business (which sees its roadmap slip and its estimates become fiction) and to the users (who see defects). The Sisyphus painting is the memory anchor for the roadmap side: the business keeps pushing the same boulder (the same feature promises) uphill and it keeps rolling back (lead times stretch, predictions fail). The beetle swarm is the anchor for the product side: debt does not merely slow development, it *leaks* quality into the shipped artifact. An exam answer that names all four labelled elements — Roadmap / Long Lead Times / Lack of Predictability on the business side, bugs as "what the users experience" on the

product side, and the central BILL paying "Technical Debt" to both — demonstrates command of the slide rather than a vague recollection of it (BeyondTechnicalDebt p.5).

Principal vs interest

What it is. This is the operationalisation of the loan metaphor into two *measurable* code properties. The deck maps the financial terms directly onto code on the **Hotspot** slide (BeyondTechnicalDebt p.12):

Loan term	Code property the deck assigns	How it's measured
Principal	Code complexity of a file/function	size, cyclomatic complexity, nesting (static metrics) — the diagram shows growing circles along a "Code Complexity" axis
Interest rate	Code change frequency	how often the file is modified in the VCS history — the diagram shows reddening circles along a "Code Change Frequency" axis

So the **principal** is *the amount you borrowed* — how much substandard complexity sits in a given file, a static property you can measure from the source alone. The **interest rate** is *how fast that debt accrues cost* — how often you are forced to come back and work in that complex code, a behavioural property you can only get from the version-control history.

What it's used for / why it matters. The crucial insight this split delivers is that **principal alone is not the cost**. A hugely complex file you never touch charges you no interest — its debt is dormant, like a loan you've forgotten about that (somehow) accrues nothing. The *actual* ongoing cost is **principal × interest rate**: complex code that you have to keep changing. This is exactly the product that defines a **hotspot** (below). The practical payoff is a *prioritisation rule*: do not rank debt by complexity (principal) alone, and do not rank it by churn (interest) alone — rank by the *product*, because that is what actually drains the team's time. It is also why a single global figure like SonarQube's "11.0% technical debt" is misleading — it sums principal across the whole codebase without weighting any of it by interest, so it cannot tell a high-cost loan from a dormant one.

When & how it's applied. Imagine two files. File A: a 4,000-line, deeply nested legacy report generator (huge principal) that has not been edited in two years (near-zero interest). File B: a 300-line authentication helper of moderate complexity (modest principal) that is touched in nearly every sprint (high interest). A naive "biggest/ugliest first" policy would refactor File A; the principal-×-interest rule correctly picks File B, because *that* is where the team is actually paying — every sprint someone wrestles with that helper, risks a security bug, and re-tests it. Refactoring File A would be satisfying but would save almost nothing; refactoring File B pays back on the very next sprint. This worked judgement is the core skill the lecture is teaching.

Hotspots — the central concept

What it is. A **hotspot** is *"complicated code that you have to work with often"* (BeyondTechnicalDebt p.12) — the single concept that ties the whole lecture together. Formally it is the *conjunction* (logical AND, or equivalently the product) of two independent measurements:

- high **code complexity** (the principal — measured statically: size, cyclomatic complexity, nesting), and
- high **change frequency** (the interest rate — measured behaviourally from the VCS: number of commits touching the file).

A file is a hotspot only when *both* are high. Complex-but-stable code is not a hotspot (dormant debt); simple-but-churned code is not a hotspot (cheap to keep changing); it is the *overlap* — complex code that is also constantly edited — that is dangerous.

What it's used for / why it matters. Hotspots are the deck's concrete answer to the Tower-of-Babel question, "where do you start paying back the debt?" (p.8). Out of thousands of files and tens of thousands of violations, the hotspot ranking gives you a *short, ordered list of the files worth caring about*. You start with the code that is both complex *and* frequently changed, because (1) that is where the interest payments are largest — the team spends the most cumulative effort there — and (2) that is where every *future* change (every **Actualization**) is most expensive and most risk-prone, so fixing it pays forward as well as back. In short, hotspots convert an overwhelming, un-prioritised debt mass into a tractable, ROI-ordered work queue.

When & how it's applied — computing and using a hotspot. Concretely, a tool like CodeScene scans the git log to count, per file, how many commits touched it (change frequency / interest) and analyses the current source to score each file's complexity (principal). It then effectively multiplies or ranks on the two together. A file with complexity score 90/100 and 400 commits ranks far above a file with complexity 90/100 and 5 commits, and above a file with complexity 20/100 and 400 commits. The top of that ranked list is the prioritised repayment plan. **Visualising hotspots:** CodeScene renders a codebase as a **nested-circle enclosure diagram** (Voronoi/circle-packing): folders nest as large pale circles, files are inner circles, **circle size $\propto$ code size/complexity** (principal), and **colour intensity (red) $\propto$ change frequency** (interest). The ReactJS map (p.13) shows packages like `react-dom`, `react-reconciler`, and `react-devtools-shared` with a few dark-red inner circles marking the true hotspots. The advantage over a static report: you instantly *see* the handful of files worth attention — the small dark-red circles — rather than scanning a flat list of thousands of violations.

Change-process link: A hotspot that falls inside the **impact set** of a change request (LO3 Impact Analysis) is a red flag — it tells you the change will land in expensive, frequently-churned, complex code, so budget extra Prefactoring and expect higher regression risk during Verification.

The hotspot slide's visual grammar (p.12)

What the slide shows. The Hotspot definition slide (BeyondTechnicalDebt p.12) is built from a framed two-row diagram that encodes the principal/interest mapping graphically, drawn over a faded hotspot map in which the file label `ActivityManagerService.java` is legible — a famously enormous class from the Android platform, used here as the background specimen of a real-world hotspot. The diagram itself reads as follows:

- **Top row — Principal.** A callout box labelled "**Principal**" points to a horizontal sequence of black-outlined circles that *grow in size* from left to right along an arrow labelled "**Code Complexity**". Size is the visual variable for complexity: the further right, the more principal the file carries.
- **Bottom row — Interest Rate.** A callout box labelled "**Interest Rate**" points to a second sequence of circles that *deepen in colour* from white through pink to dark red along an arrow labelled "**Code Change Frequency**". Colour saturation is the visual variable for churn: the redder, the more often the file is touched.
- **Convergence — Hotspot.** Both arrows point rightward to a single large, solid dark-red circle labelled "**Hotspot**" — the graphical statement that a hotspot is what you get where *maximum size meets maximum redness*, i.e. high complexity AND high change frequency together (BeyondTechnicalDebt p.12).

Why this matters for reading every later slide. This slide establishes the *visual grammar* that every subsequent CodeScene screenshot in the deck uses: in the ReactJS map (p.13), the checker.ts context map (p.15), and the X-Ray entry map (p.17), **circle size always means complexity/size (principal)** and **red colour always means change frequency (interest)**. If you internalise the two-row diagram on p.12, every later map becomes instantly legible — and you avoid the classic misreading (flagged under Pitfalls) of

treating the *biggest* circle as the priority when the priority is the *big-and-dark-red* circle. The italicised definition line above the diagram — "*A hotspot is a complicated code that you have to work with often*" — is the sentence to quote verbatim in any exam answer (BeyondTechnicalDebt p.12).

The ReactJS hotspot map — complete package inventory (p.13)

What the slide shows. Page 13 (titled simply "**ReactJS**") is a full-screen CodeScene enclosure map of the React repository, demonstrating the hotspot visualisation at real-codebase scale. The outermost pale circle is the repository; within it, each large translucent circle is a package directory, and the small circles inside those are files, sized by complexity and coloured by change frequency per the p.12 grammar. The package labels legible on the slide, enumerated completely, are: **eslint-plugin-react-hooks**, **react-interactions**, **react-debug-tools**, **react-refresh**, **react-is**, **react-native-renderer**, **shared**, **react**, **legacy-events**, **scheduler**, **react-dom**, **react-reconciler**, and **react-devtools-shared** (BeyondTechnicalDebt p.13).

Where the heat is. Most of the map is pale — the overwhelming majority of React's files are *not* hotspots, which is itself the lesson: even in a heavily-developed, world-scale codebase, the expensive debt concentrates in a handful of files. The visible dark-red concentrations cluster in three packages: **react-dom** (several red files on the right), **react-reconciler** (a cluster of red circles — the reconciliation engine is where React's essential complexity lives and where change concentrates), and **react-devtools-shared** (containing some of the darkest, most prominent hotspot circles on the map — and, not coincidentally, the package the deck drills into with X-Ray on p.17–18) (BeyondTechnicalDebt p.13, p.17).

How to use this in an answer. If asked to "describe how a behavioural tool presents a large codebase," narrate this map: thousands of files reduced to one screen; size = complexity, red = churn; the eye goes straight to the few dark-red circles in **react-reconciler** and **react-devtools-shared**; and that handful — not SonarQube's 10,072 violations (p.7) — is the work queue. The map is the answer to the Tower-of-Babel question "where do you start?" (p.8): you start at the darkest red circle (BeyondTechnicalDebt p.8, p.12–13).

Hotspots are also common sources of bugs (empirical grounding)

What it is. This concept supplies the *evidence* that the hotspot heuristic is not just intuitive but empirically validated. Slide 14 (BeyondTechnicalDebt p.14) backs it with research: **Graves, Karr, Marron & Siy, Predicting Fault Incidence Using Software Change History** (IEEE Transactions on Software Engineering, 2000), a study that built statistical models to predict which modules of a large system would contain faults. Two findings are quoted:

- "*In general, process measures based on the change history are more useful in predicting fault rates than product metrics of the code: For instance, the number of times code has been changed is a better indication of how many faults it will contain than is its length.*" — i.e. **change frequency beats size** as a fault predictor (this is precisely the *interest-rate* axis of a hotspot).
- "*...the older module will have roughly a third fewer faults...*" and the most successful model measured a module's fault potential as a **weighted sum of all its past changes, with large, recent changes weighted most heavily**. — i.e. **recent churn matters most**; a module that was changed a lot *recently* is more fault-prone than one that was changed a lot long ago and has since stabilised.

What it's used for / why it matters. This is the deck's defence against the obvious objection that "complexity × change frequency" might be a plausible-sounding but unfounded rule. The Graves result shows that the *change-history* dimension — the very thing static analysers ignore and behavioural analysers add — is independently a *better fault predictor than size or other product metrics*. That matters for two reasons: first, it justifies putting **change frequency** at the centre of debt prioritisation (the interest-rate axis is also a bug-rate axis); second, it means hotspots are not only where you spend the most maintenance effort but also where

defects concentrate — so repaying hotspot debt is simultaneously the highest-ROI *quality* (bug-reduction) move, doubling the return.

When & how it's applied. When deciding where to focus scarce QA and refactoring effort, this finding tells you to weight by *recent change activity*, not by file size or age. A team that ranks modules by "biggest, so probably buggiest" is using the *weaker* predictor; ranking by "most-changed, especially most-recently-changed" uses the stronger one. In practice you take your hotspot list and trust that those files are also your likely bug factories, so you (a) refactor them, (b) point your strongest regression tests at them, and (c) review changes to them more carefully — all justified by the same empirical result. It also explains why a recently-stabilised but historically-churned file can be de-prioritised: its *recent* churn is low, so its predicted fault rate has fallen.

The Graves et al. paper — full bibliographic record (p.14)

The citation as the slide presents it. Slide 14 (headline "*Hotspots are also common sources of bugs*") reproduces the header of the paper itself — "*Predicting Fault Incidence Using Software Change History*" — in the style of a research-portal listing: *Article (PDF Available)* in **IEEE Transactions on Software Engineering** **26(7):653–661, August 2000**, with a reads counter (~420 reads) (BeyondTechnicalDebt p.14). The four authors are shown with affiliations: **Todd L. Graves, Alan F. Karr** (RTI International), **J. S. Marron**, and **Harvey Siy** (University of Nebraska at Omaha) (BeyondTechnicalDebt p.14). Being able to name "Graves et al., IEEE TSE, 2000" precisely is a cheap way to earn credibility in an exam answer about the empirical basis of hotspots.

The two quoted findings, verbatim with the slide's emphasis. Below the paper header the slide quotes two passages, each carrying its own bolding:

1. "*In general, process measures based on the change history are more useful in predicting fault rates than product metrics of the code: For instance, the number of times code has been changed is a better indication of how many faults it will contain than is its length.*" (BeyondTechnicalDebt p.14)
2. "*We also compare the fault rates of code of various ages, finding that if a module is, on the average, a year older than an otherwise similar module, the older module will have roughly a third fewer faults. Our most successful model measures the fault potential of a module as the sum of contributions from all of the times the module has been changed, with large, recent changes receiving the most weight.*" (BeyondTechnicalDebt p.14)

Unpacking into four separable claims. For exam purposes the two quotations decompose into four distinct, individually testable claims: (a) **process beats product** — measures derived from the *change history* (process) outpredict measures derived from the *code itself* (product) for fault rates; (b) **changes beat length** — specifically, the *count of changes* to a module predicts its fault content better than its *size*; (c) **age protects** — an otherwise-similar module a year older carries roughly **one-third fewer faults**, i.e. code that has survived unchanged has been hardened by exposure; and (d) **the best model is a weighted sum** — fault potential = the sum of contributions from *all* past changes, with **large, recent changes weighted most heavily** (BeyondTechnicalDebt p.14). Claim (d) is the most precise and the most examinable: it says the right predictor is neither raw churn nor recency alone but a *decaying, size-weighted accumulation* of the change history — which is exactly the quantity a behavioural tool computes and a static analyser cannot, since the information lives only in the version-control system (BeyondTechnicalDebt p.10–11, p.14).

Worrisome trends — reading a hotspot over time

What it is. Because CodeScene is a *movie* and not a snapshot, it can show not just a file's *current* state but the *trajectory* of its complexity over time — and flag **worrisome trends** where that trajectory is heading the

wrong way (BeyondTechnicalDebt p.15). The worked example is TypeScript's `src/compiler/checker.ts`, presented with a dashboard of facts plus a trend chart:

- **Snapshot facts:** **Size** 25,774 lines of code; **Change Frequency** 785 commits; **Main Author** Anders Hejlsberg (26%); **Knowledge Loss** 0% (no abandoned code); **Defects** 1,051 (133% bug fixes — i.e. more bug-fix commits than features); **Code Health** 1 (the lowest band, shown in red).
- **Complexity Trend chart:** a **rising red complexity curve** climbing toward ~130,000 while the **blue lines-of-code curve stays nearly flat** — complexity is growing *faster than* the file is growing. That divergence (complexity outpacing size) is the warning sign: the file is getting *harder per line*, the classic signature of accumulating principal in a high-interest file.

What it's used for / why it matters. A single snapshot metric ("this file has complexity X") cannot tell you whether the debt is *being managed or compounding* — two files with the same current complexity are in completely different situations if one is stable and the other is climbing steeply. The trend view exists to answer the dynamic question: *is this getting better or worse?* The decisive signal it surfaces is **complexity rising while LOC stays flat**, which means the team is adding tangle (branches, special cases, nesting) without adding much code — they are *concentrating* debt. Catching that early lets a team intervene *before* a file crosses the threshold into unmanageable, rather than after, when a rewrite is the only option. It turns debt management from reactive (fix the worst file) into proactive (stop the climbing file before it becomes the worst).

When & how it's applied. You open the top hotspot from your ranking and look at its complexity-over-time chart. If complexity tracks LOC (both rising together), the file is just *growing* — possibly fine. If complexity climbs while LOC is flat (as with `checker.ts`), the file is *rotting in place* and you act now: schedule a Prefactoring pass, tighten review on further changes, and consider splitting it. The supporting numbers sharpen the read: 785 commits confirm high interest, "133% bug fixes" confirms it is actively causing defects, and Code Health 1 confirms it is already in the danger zone. The lesson is to act on *direction and slope*, not just the current dot.

The checker.ts dashboard, field by field (p.15)

Breadcrumb and context map. The "Worrisome Trends" slide (BeyondTechnicalDebt p.15) shows a CodeScene file dashboard whose breadcrumb reads `TypeScript / TypeScript / src / compiler / checker.ts` — the TypeScript compiler's type checker. To its left sits the hotspot context map: `checker.ts` rendered as one enormous dark-red circle that dwarfs every neighbour in the `compiler` package (the labels `transform...` and `commandLineParser.ts` are legible beside it as small pale circles) — a single file so dominant it visually is the package's debt (BeyondTechnicalDebt p.15).

Every dashboard field, with its reading. The panel lists, top to bottom:

Field	Value on slide	What it tells you
Size	25,774 Lines of Code	The file is enormous — high principal, and far too big to "just refactor" (motivating X-Ray, p.16–17).
Code Health	1 (red badge)	CodeScene's per-file quality score at its worst band — the file is already in the danger zone.
Change Frequency	785 Commits	Very high interest rate: this complex file is constantly worked on.
Main Author	Anders Hejlsberg (26%)	The largest single contributor holds about a quarter of the authorship — knowledge is spread, not monopolised.

Field	Value on slide	What it tells you
Knowledge Loss	0% Abandoned Code	No code in the file belongs to departed authors — the <i>social</i> debt here is currently nil, in contrast to its severe <i>technical</i> debt.
Defects	1,051 (133% Bug Fixes)	Over a thousand defect-linked changes; the bug-fix percentage exceeding 100% signals that defect-fixing activity dominates the file's change profile.
Last (Modified)	"0 months ago" (the field is partially cut off at the slide's bottom edge)	The file is being modified <i>right now</i> — its interest is accruing in the current period, the heaviest weight in the Graves model (p.14).

(All values: BeyondTechnicalDebt p.15.)

The Complexity Trend chart, precisely. Below the map sits the **Complexity Trend** chart, captioned "*Click on a point to diff the code changes*" — every plotted point is a real commit you can open. The **x-axis runs from October 2015 to April 2019** in quarterly ticks (October 2015, April, July, October 2016, ... April 2019); the **y-axis (Complexity, ws)** is scaled in thousands up to ~130,000. Three series are drawn: the **red Complexity line**, climbing steadily and steeply to ~130,000; the **blue Lines of Code line**, nearly flat far below it (~25–30k, consistent with the 25,774 LOC figure); and a **green Code Comments line** flatter still along the bottom (BeyondTechnicalDebt p.15). The damning read is the *divergence*: over three and a half years complexity quintupled-plus while the line count barely moved — the team has been packing ever more branching and nesting into the same physical span of code, the textbook signature of compounding principal. The flat comments line adds a quiet third observation: documentation is not growing to compensate (BeyondTechnicalDebt p.15).

Why this file is the perfect specimen. `checker.ts` shows that a file can be catastrophic on the technical axes (Code Health 1, 785 commits, 1,051 defects, runaway complexity trend) while perfectly healthy on the social axis (0% knowledge loss, active well-known authorship). The deck's later React example inverts this profile — see the next deep-dive entry point (p.17), where the technical numbers are milder but 80% of the file is one person's knowledge. Holding both specimens in mind covers both halves of "technical debt isn't just technical" (BeyondTechnicalDebt p.15, p.17, p.20).

X-Ray — drilling from file to function (deep dive)

What it is. X-Ray is CodeScene's "deep dive" capability (BeyondTechnicalDebt p.16, "Deep Dive") that applies the *same* hotspot logic one level down — *inside* a single problematic file — to rank its individual **functions**. The motivation is stated on p.17: "*We've identified a problematic file but it's still huge! Let's X-Ray it to find the most problematic functions.*" A hotspot file can be tens of thousands of lines, far too large to "just refactor"; X-Ray finds the specific functions within it that carry the complexity and the churn. The X-Ray function table (p.18, a React `attach.*` module) ranks functions by **Change Frequency**, **Lines of Code**, and **Cyclomatic Complexity** (colour-coded yellow→red):

Function	Change Freq	LOC	Cyclomatic Complexity
attach (top-level context)	103	109	9
attach.recordMount	37	48	8
attach.handleCommitFiberRoot	32	89	19
attach.flushPendingEvents	30	105	15
attach.inspectElement	30	93	12
attach.updateFiberRecursively	24	220	48

Function	Change Freq	LOC	Cyclomatic Complexity
<code>attach.inspectElementRaw</code>	24	180	45

What it's used for / why it matters. X-Ray exists to make a refactoring target *precise and affordable*. Knowing "`checker.ts` is a hotspot" is not actionable when the file is 25k lines — that is still "boil the ocean." X-Ray converts the file-level verdict into a function-level work item: the same principal × interest logic that ranked files now ranks functions, so you can see that within this module `updateFiberRecursively` (cyclomatic complexity 48) and `inspectElementRaw` (complexity 45) are the function-level hotspots to refactor first. Instead of rewriting a 2,000-line file you fix the two functions doing the damage. That precision is what makes the work *estimable, reviewable, and safe* — and it is exactly what scopes a realistic **Prefactoring/Postfactoring** task.

When & how it's applied — reading the table. You read the table by combining the three columns the same way you read a file-level hotspot. A high *cyclomatic complexity* (48, 45 — flagged red) means the function has many independent paths, so it is hard to understand and hard to test exhaustively (high principal); a high *change frequency* means it is repeatedly edited (interest); *LOC* contextualises both. Note that the top row by change frequency (`attach` top-level, 103) is *not* the worst target — its complexity is only 9, so it is cheap to keep changing. The genuine refactoring targets are the rows where complexity is red *and* the function is non-trivially changed: `updateFiberRecursively` and `inspectElementRaw`. You would scope a Prefactoring task to decompose exactly those two functions (extract methods, flatten nesting, reduce branching), re-test, and move on — a bounded, affordable fix rather than an open-ended file rewrite.

The X-Ray entry point — the react-devtools-shared backend file (p.17)

What the slide shows. Page 17 is the bridge between the file-level and function-level analyses: the left half is the React hotspot map zoomed into the dark-red target file, the middle is that file's CodeScene dashboard, and the right half carries the slide's only text — "*We've identified a problematic file but it's still huge! Let's X-Ray it to find the most problematic functions.*" (BeyondTechnicalDebt p.17). The breadcrumb reads `react / react / packages / react-devtools-shared / src / backend` — i.e. the deep-dive target lives in the `react-devtools-shared` package that the p.13 map already showed glowing dark red (BeyondTechnicalDebt p.13, p.17).

Every dashboard field on p.17. The panel reports:

Field	Value on slide	Reading
Size	2,444 Lines of Code	An order of magnitude smaller than <code>checker.ts</code> (25,774), yet still "huge" by refactoring-task standards — hence X-Ray.
Code Health	amber/yellow badge	Degraded but not the worst band — contrast <code>checker.ts</code> 's red 1 (p.15).
Change Frequency	187 Commits	High interest for a single file, though far below <code>checker.ts</code> 's 785.
Main Author	Brian Vaughn (80%)	Four-fifths of the file is one developer's work — extreme knowledge concentration.
Knowledge Loss	6% Abandoned Code	A small fraction already belongs to departed authors.
Defects	24 (20% Bug Fixes)	A modest defect count; one in five changes is a bug fix.
Last Modified	6 months ago	Recent enough to matter, though not under active churn at capture time.

Below the panel sit four **action buttons** — **View Code, X-Ray, Trends, Code Review** — the operations CodeScene offers on a selected file (the deck clicks **X-Ray**), and beneath them a rising **Complexity Trend** thumbnail confirming the file's principal is growing (BeyondTechnicalDebt p.17).

The thread this slide ties. The main-author figure is the deck's hidden continuity device: **Brian Vaughn (80%)** here is *the same developer* whose departure is simulated in the off-boarding analysis on p.22–23. The deck is quietly showing you a file that is simultaneously a *technical* hotspot (complex, churned, X-Ray-worthy) and a *social* risk (one person owns 80% of it) — and then, five slides later, showing you exactly what happens to the surrounding packages when that person "leaves" and the map turns red (BeyondTechnicalDebt p.17, p.22–23). Citing this cross-slide link is a high-value exam move: it demonstrates that the two halves of the lecture (hotspots; legacy/off-boarding) are two lenses on the *same* code.

The complete X-Ray function table (p.18)

The full table, all visible rows. The X-Ray results slide (BeyondTechnicalDebt p.18) shows a sortable table — two tabs at the top read **"Structural Recommendations"** and **"Change Frequency Distribution"** — listing the functions of the `attach` module with four data columns: **Change Frequency**, **Lines of Code**, **Cyclomatic Complexity** (cell-shaded yellow or red by severity), and **Overloaded Functions?**. Every row also carries two per-row action buttons (a trend-chart icon and a view-code icon). The complete visible contents, in the slide's own descending change-frequency order:

Function	Change Frequency	Lines of Code	Cyclomatic Complexity	Overloaded Functions?
<code>attach</code> (top-level context)	103	109	9 (yellow)	1
<code>attach.recordMount</code>	37	48	8 (yellow)	1
<code>attach.handleCommitFiberRoot</code>	32	89	19 (red)	1
<code>attach.flushPendingEvents</code>	30	105	15 (red)	1
<code>attach.inspectElement</code>	30	93	12 (yellow)	1
<code>attach.flushInitialOperations</code>	29	51	6 (yellow)	1
<code>attach.updateFiberRecursively</code>	24	220	48 (red)	1
<code>attach.inspectElementRaw</code>	24	180	45 (red)	1
<code>attach.recordUnmount</code>	18	50	9 (yellow)	1

(All values: BeyondTechnicalDebt p.18.)

Reading the columns together. The table is *sorted by change frequency*, but the refactoring priority comes from reading frequency *against* complexity, exactly as at file level: the top row (`attach` top-level, 103 changes) has complexity only 9 — high interest on a small principal, cheap to keep paying — while `updateFiberRecursively` (48) and `inspectElementRaw` (45) sit lower in the sort yet are the true targets because they combine red-flagged complexity with non-trivial churn (24 changes each) and the largest bodies (220 and 180 LOC). The four red-shaded cells (19, 15, 48, 45) mark the file's entire genuinely dangerous surface: four functions out of a 2,444-line module (BeyondTechnicalDebt p.17–18). The **Overloaded Functions?** column reads 1 for every visible row — one definition per name, so no complexity is hiding behind overload resolution here; the column exists because X-Ray must aggregate metrics across same-named

overloads when they occur (BeyondTechnicalDebt p.18). The two tabs matter too: **Structural Recommendations** is where the tool proposes refactorings, and **Change Frequency Distribution** shows how edits spread across the module — both reinforcing that X-Ray is an *analysis workbench*, not just a ranked list (BeyondTechnicalDebt p.18).

Change coupling

What it is. Change coupling (BeyondTechnicalDebt p.19) is a *temporal* dependency between files (or functions): the property that they **tend to change together in the same commits**, regardless of whether the source code shows any direct reference between them. X-Ray surfaces it as a **chord diagram** — a circle with arcs connecting entities that co-change (e.g. `renderer.js` ↔ `attach (top-level context)`); the thicker/more-present the arc, the more often the two are edited in lockstep. Critically, this is mined from the *version-control history*, not from the code structure, so it catches relationships that are invisible to any static analysis: two files with no import, no call, and no shared symbol that nonetheless *always* change together because they implement two halves of one concept.

What it's used for / why it matters. Change coupling has two distinct uses. First, **as a debt smell**: high change coupling means a single logical change is **smeared across multiple files** — you cannot make one conceptual edit in one place; you must remember to edit three or four files in concert, and if you forget one you ship a bug. That is precisely Fowler's *Shotgun Surgery* smell ([Fowler99]), and indeed "Shotgun surgery" appears explicitly in the deck's "much more" list (p.24). It signals that the design has split something that should be cohesive, and it is itself a form of debt worth repaying (by re-grouping the co-changing logic). Second, **as an impact-analysis input**: if two files have historically always changed together, then changing one is strong *evidence* you will need to change the other — even when no static dependency exists between them.

When & how it's applied. Reading the chord diagram, you look for entities tied by strong arcs to the file you are about to modify. Suppose you are about to change `attach`; the diagram shows a heavy arc to `renderer.js`. That tells you two things: (1) for *this* change, `renderer.js` almost certainly belongs in your **impact set**, so you plan to inspect and probably edit and re-test it too — even though a static call-graph search might never have flagged it; and (2) the persistent coupling itself is a design defect (Shotgun Surgery) you might schedule for repayment by consolidating the two halves. In the change process, this makes change coupling a direct, *evidence-based* correction to static impact analysis, which systematically *under-counts* the impact set by missing exactly these hidden temporal dependencies.

Reading the chord diagram, entity by entity (p.19)

What the slide shows. The Change Coupling slide (BeyondTechnicalDebt p.19) is a full-screen X-Ray **chord diagram**: a circle whose circumference is divided into arcs, one per entity (note that the entities here are *functions within named files*, not just files — X-Ray couples at function granularity), with ribbons ("chords") drawn across the circle's interior connecting entities that change together in the same commits. The labels legible around the rim include: **`renderer.js - attach (top-level context)`** (the highlighted, bold entity — the same `attach` module from the p.18 table), **`CommitTreeBuilder.js - updateTree.switch`** (its rim arc rendered magenta), and faded entries for **`CommitTreeBuilder.js - updateTree (top-level context)`**, **`SelectedElement.js - SelectedElement`**, **`renderer.js - attach.flushPendingEvents`**, and an **`agent.js`** entity (BeyondTechnicalDebt p.19).

The visual encoding. Selected/active couplings are drawn as thick, saturated blue-violet ribbons; non-selected couplings recede to faint grey. On the slide, two prominent ribbons emanate from `renderer.js` `attach (top-level context)`: one sweeping across the circle to the magenta-arc'd `CommitTreeBuilder.js`

`updateTree.switch`, and one descending to another `attach` entity at the bottom of the circle. The reading rule: **a ribbon between two arcs = those two entities co-change; the ribbon's prominence = the strength/recency of the coupling** (BeyondTechnicalDebt p.19).

What the specific couplings mean. The strongest visible coupling — the `attach` function in `renderer.js` co-changing with `updateTree.switch` in `CommitTreeBuilder.js` — is exactly the kind of relationship no static analyser would rank highly: two functions in *different files* that the commit history proves are two halves of one concept (changes to how the renderer attaches/records its tree force matching changes in how the commit-tree builder updates its tree). For a developer about to modify `attach`, the diagram is a pre-computed warning: budget for `CommitTreeBuilder.js` in the impact set, and treat the persistent cross-file lockstep as a *Shotgun Surgery* design smell worth consolidating (BeyondTechnicalDebt p.19, p.24; [Fowler99]). The diagram thereby serves both of change coupling's uses at once — impact-analysis input and debt smell — on one screen.

Legacy code — why debt isn't just technical

What it is. The final section (BeyondTechnicalDebt p.20, "*Legacy Code — Why technical debt isn't just technical*") widens debt from a purely *code* property to its **social / organisational** dimension, and in doing so redefines "legacy code" along two axes (p.21):

- **lacks quality** — but only in a *relative* (relative-perspective) sense; "legacy" is a judgement made from a viewpoint, and one team's legacy is another team's perfectly normal system. Quality alone does not make code legacy.
- **we didn't write ourselves** — the ownership / knowledge dimension. Code we inherited (because its author left, or because we took over the project) is "legacy" to *us* even if it is well written, because we lack the mental model of *why* it is the way it is.

What it's used for / why it matters. This reframes debt as partly a *people* problem rather than a pure code-quality problem. The key consequence is that code becomes "legacy" not only when it is *bad*, but when **the knowledge of it leaves** — when the person who held the design rationale in their head walks out the door. That is why the deck pairs this with "**The Technical Debt That Wasn't**" (p.21): a panel of three products, two with clear owners (thumbs-up) and one with no owner (a "?") — abandoned code whose risk is **organisational**, not in the code itself. The code might be fine, but with no one who understands it, every change to it is dangerous. This matters because it reveals a class of risk that *no static quality tool can see*: a clean, well-tested file can still be a serious liability if its only knowledgeable author has left. The whole point — "*technical debt isn't just technical*" — is to make teams manage ownership and knowledge as deliberately as they manage code complexity.

When & how it's applied. The social dimension is *measurable from the VCS*: who authored each line, who is still on the team, and how much code has become **abandoned** (zero current owners). This is the same **Knowledge Loss** field seen on the trend slide (p.15, where `checker.ts` showed 0% knowledge loss because its author was still active). In practice a team uses this to find code that is "legacy in waiting" — files concentrated in one author's history — and to find truly abandoned code (high knowledge loss) where the next change will be slow and risky because nobody understands it. The remedy is organisational, not technical: spread ownership, pair on the lonely files, and write down the rationale *before* the knowledge leaves.

Measuring & simulating off-boarding risk

What it is. Because authorship is recorded in the VCS, CodeScene can do something predictive: it can answer "*How quickly can you turn your codebase into legacy code?*" (BeyondTechnicalDebt p.22) by **simulating off-boarding** — modelling what *would* happen to the codebase's knowledge distribution if one or more

specific developers left. You pick a developer (the example uses "Brian Vaughn") and the tool re-colours the map to show which code would suddenly have *no remaining knowledgeable owner*. The map's **legend** distinguishes several states: **Knowledge** (blue = still owned by someone present), **Current Loss** (already abandoned today), **Simulated Loss** (would become abandoned in the scenario), **Off-Boarding Risk** (red = code at risk if the selected person leaves), and **Inconclusive**. The before/after pair makes the effect visual: *before* (p.22) the `react` directory glows blue (knowledge present); *after they leave* (p.23) large regions turn red — code that would instantly become **legacy / off-boarding risk** if that author departed.

What it's used for / why it matters. This is the social half of debt made *quantitative and predictive* — it turns the vague worry "we'd be in trouble if so-and-so quit" into a concrete map of *exactly which* parts of the system depend on a single person. It is essentially a **bus-factor** analysis (how many people would have to be "hit by a bus" before a part of the system becomes unmaintainable) computed automatically from authorship history. Its value is that it lets an organisation *see and act on* knowledge concentration *before* the knowledge actually walks out the door — converting a hidden organisational risk into a managed one. Knowledge loss, unlike code complexity, cannot be refactored away after the fact; once the person is gone the knowledge is gone, so the only effective intervention is *advance* mitigation, which requires *advance* visibility.

When & how it's applied. A team lead runs the off-boarding simulation for each key developer (especially anyone known to be leaving, or any single-author region the legacy analysis flagged). Wherever the map lights up red — large regions that would become legacy if one person left — the lead schedules concrete mitigations *while that person is still present*: pair-programming sessions to transfer the mental model, written documentation of the design rationale, and deliberate redistribution of ownership (assigning others to make changes in that region so the authorship history broadens). Done before an actual departure, this shrinks the red regions and protects the team's ability to keep maintaining that code. Done as part of off-boarding planning, it produces a prioritised hand-over checklist focused exactly on the code only the departing person understood.

The off-boarding simulation UI in detail (p.22–23)

Titles and instruction text, verbatim. The simulation spans two slides. The "before" slide is headlined **"How quickly can you turn your codebase into legacy code?"** and both slides carry the same instruction line: *"Simulate the effects of a planned off-boarding if some developers leave your organization"* (BeyondTechnicalDebt p.22–23). The "after" slide is headlined **"After they leave ..."** (BeyondTechnicalDebt p.23). The phrase *planned off-boarding* matters: the tool is positioned for *foreseen* departures (resignation notice, team restructuring), where there is still time to act.

The legend — all five knowledge states. Both slides share a five-entry legend that defines the complete vocabulary of the knowledge map (BeyondTechnicalDebt p.22–23):

Legend state	Colour on map	Meaning
Knowledge	blue	Code whose authors are still present — healthy, owned code.
Current Loss	black	Code <i>already</i> abandoned today (authors already gone) — existing knowledge debt.
Simulated Loss	red (fill)	Code that <i>would become</i> abandoned under the simulated departure — the scenario's direct casualty.
Off-Boarding Risk	dark red	Code at risk if the selected person leaves — the warning band around the scenario.
Inconclusive	unfilled/white	Code where authorship signal is insufficient to classify.

The distinction between **Current Loss** (a fact about today) and **Simulated Loss** (a prediction about the scenario) is the legend's examinable core — conflating them is a marked error, because one calls for *recovery* measures (the knowledge is already gone) and the other for *prevention* (the knowledge can still be transferred).

The workflow as the UI presents it. The right-hand panel is headed **Developers** with the prompt "*Select one or more developers to simulate their departure from your organization*" and a "**Filter developer names:**" search box — on the slide the string "**brian**" has been typed, surfacing the checkbox for **Brian Vaughn** (BeyondTechnicalDebt p.22). A directory selector shows **Directory: react** — the simulation is scoped to the React repository's tree. On the before slide, with no one off-boarded, the `react` directory renders almost uniformly **blue** (knowledge present). On the after slide, the panel lists "**Simulated offboarded authors: Brian Vaughn**" (checked), and the same map now shows **red regions scattered across the packages** — every area where Vaughn was the effective sole owner has flipped to Simulated Loss / Off-Boarding Risk (BeyondTechnicalDebt p.23). Recall from p.17 that Vaughn owned **80%** of the `react-devtools-shared` backend hotspot: the simulation generalises that single-file observation to the whole repository in one picture (BeyondTechnicalDebt p.17, p.22–23).

Why the UI detail is worth knowing. Exam questions about this material tend to ask either "what does the tool let you do?" (answer: select developers, scope a directory, and re-render the knowledge map under their simulated departure) or "what do the colours mean?" (answer: the five legend states above). Knowing that the example developer is the same person as the p.17 hotspot's main author turns a tool description into an argument about why *technical* and *social* debt must be managed together (BeyondTechnicalDebt p.17, p.20–23).

Causes of technical debt (synthesised from the deck)

What it is. This is a consolidation — drawn together by this guide from points the deck makes across its slides — of *why* technical debt accumulates in the first place. It is not a single deck slide but a synthesis of the deck's recurring themes:

- **Software evolution itself** — Lehman's Continuing Change + Increasing Complexity make rising complexity the *default* trajectory of any living system (p.3). This is the structural, unavoidable cause: you must keep changing the system, and changing it tends to complicate it.
- **Ignoring complexity / schedule pressure** — taking shortcuts to hit a deadline that then compound and slow every sprint (Hickey, p.4). This is the *deliberate-loan* cause: debt taken on knowingly (or carelessly) to ship faster now.
- **Age and historical accretion** — "*Thousands of years of technical debt*" (Tower of Babel, p.8): systems accumulate so much debt over so long that the dominant problem becomes simply **knowing where to start** paying it back.
- **Knowledge loss / staff turnover** — code becoming "legacy" because its authors leave, taking the design rationale with them (p.20–23). This is the *social* cause: debt that appears not because the code changed but because the *people* did.
- **Hidden coupling** — changes that spread across co-changing files (Shotgun Surgery), invisible to static tools, so the design degrades without any single edit looking wrong (p.19).

What it's used for / why it matters. Cataloguing the causes matters because *different causes need different remedies*, and the deck's whole prescriptive arc maps onto them. Evolutionary/structural debt (Lehman) calls for a *standing* refactoring budget; deliberate shortcut debt (Hickey) calls for *discipline and trade-off awareness* at the point of borrowing; accretion debt (Tower of Babel) calls for *prioritisation* (hotspots) rather than blanket repayment; knowledge-loss debt calls for *organisational* mitigation (off-

boarding management); and coupling debt calls for *design* repair (consolidating co-changing logic). Recognising which cause is dominant for a given liability tells you which tool in the lecture to reach for.

When & how it's applied. Facing a debt-laden module, a team asks *why* it is the way it is. If it is just old and heavily evolved, expect Lehman and budget recurring cleanup. If it is full of obvious shortcuts from a past crunch, the borrowing was deliberate and the fix is a focused repayment. If nobody understands it, the cause is knowledge loss and the first step is *recovering* understanding (often via the very VCS history and authorship maps this lecture describes), not blindly rewriting. Diagnosing the cause prevents the common mistake of applying a code remedy to a people problem (or vice versa).

Measurement — conventional snapshot vs behavioural movie

What it is. This concept is the deck's *central methodological contrast*: two fundamentally different philosophies for *measuring* technical debt.

(a) Conventional static analysis — a *snapshot* (BeyondTechnicalDebt p.6–8). A static analyser inspects the *current* source code at a *single point in time* and reports counts of violations and a derived debt figure. The "Actionable?" Sonar-style dashboard (p.7, source: austinjug.org *Heinz Technical Debt Sonar*) is the deck's specimen: it reports a large, precise-looking pile of numbers — **162,306** lines of code, **1,447** classes, **26.6%** comments, **7.1%** duplications, complexity **3.1/method**, **30.9/class**, **42.2/file**, rules compliance **83.7%**, **10,072** violations (incl. **8,794 Major**), and a headline **Technical Debt: 11.0% / €341,563 / 683 man days**. Every number is exact. But the slide's title is literally "Actionable?" and p.6 asks "What is missing?" — the deck's critique is that this snapshot tells you *how much* debt exists but not **where to start**, and crucially that **static analysis can never tell you if excess complexity actually matters** (p.10), because it has no information about how the code is *used* or *worked on*.

(b) Behavioural / evolutionary analysis — a *movie* (BeyondTechnicalDebt p.9–11). Instead of one frame, this approach mines the *whole history* and the *people* dimension that the snapshot lacks. CodeScene's pipeline (p.11): inputs are **Source Code + Version-Control Data (git) + Project-Management Tools (e.g. JIRA)** → **Code, Process and Evolutionary Metrics** → **Pattern Detectors, Machine Learning and Intelligence** → **Visualizations, priorities, and predictive analytics**. The two added dimensions are explicitly + **Time aspect** (how the code evolved) and + **Organization & people** (who works on it, how, how often) (p.10). The result is not a flat figure but a *prioritised, ranked* view — hotspots first — instead of a list of 10,072 violations with no ordering.

What it's used for / why it matters. The contrast is used to justify the entire methodological turn of the lecture: *why bother with behavioural analysis when static tools already give you a debt number?* The answer is that the static number, however precise, is **not actionable** — precision is not the same as usefulness. A snapshot can tell you a building has 10,072 problems; it cannot tell you which room is on fire. The behavioural movie adds exactly the missing dimension — *how people interact with the code over time* — that lets you separate the debt that is costing you (high-interest, frequently-worked code) from the debt that is dormant. This is the difference between *measuring the code* and *measuring how people work with the code*, and only the latter answers "where do we start?"

When & how it's applied. A team that has only a Sonar report knows it has "11% debt / 683 man-days" but has no rational starting point — repaying 683 man-days of work blindly is the Tower-of-Babel trap. They apply the behavioural approach by feeding their git history (and optionally JIRA) into a tool like CodeScene, which produces the hotspot ranking, trends, X-Ray drill-downs, and ownership maps described throughout this lecture. They then start at the top of that ranking. The exam contrast in one line: **static analysis measures the code; behavioural analysis measures how people work with the code** — and it is the latter that tells you which debt is actually costing you interest.

The "Actionable?" Sonar dashboard, panel by panel (p.7)

The full inventory. The deck's specimen of conventional static analysis (BeyondTechnicalDebt p.7; the slide's source line reads <http://www.austinjug.org/presentations/HeintzTechnicalDebtSonar.pdf>) deserves a complete walk-through, because its sheer density is the argument — every panel is precise, and none of it answers "where do we start?". Reading the dashboard left-to-right, top-to-bottom:

- **Lines of code: 162,306** — with sub-figures **325,036 lines** (total physical lines), **87,758 statements**, and **1,060 files**.
- **Classes: 1,447** — with **103 packages**, **14,271 methods**, and **+1,262 accessors**.
- **Comments: 26.6%** — **58,991 lines** of comments, **59.1% documented API**, **5,418 undocumented API** entries, **1,164 commented-out LOCs**.
- **Duplications: 7.1%** (highlighted on the slide) — **22,906 duplicated lines** in **566 blocks** across **174 files**.
- **Complexity: 3.1 per method, 30.9 per class, 42.2 per file** — total **44,773**, with a histogram distributing complexity over methods and classes.
- **Rules compliance: 83.7%**, accompanied by a spider/radar chart of the **SIG Maintainability Model** with four labelled axes: **(A)nalsability**, **(C)hangeability**, **(S)tability**, **(T)estability** — a maintainability-quality model scoring the system on how easy it is to analyse, change, stabilise, and test.
- **Violations: 10,072** — broken down as **Blocker 0**, **Critical 0**, **Major 8,794 (circled on the slide)**, **Minor 65**, **Info 1,213**.
- An active **alert banner**: *"Duplicated lines (%) > 5"* (orange) — the only rule currently tripping an alarm.
- **Tags: 356** — **0 mandatory**, **356 optional**, with a pie of tag kinds (FIXME / TODO / @deprecated-style markers).
- **Events panel**: 2010-07-26 — Version 6.x; 2009-06-07 — Version 6.0.x; 2009-02-15 — Alert Orange — release and alert history.
- **Technical Debt: 11.0%** (circled), valued at **341,563** (the dashboard prints the figure with a dollar sign, **\$ 341,563**) and **683 man days**, with a pie attributing the debt to **Duplication**, **Violations**, **Complexity**, and **Comments**.
- The project identity line: **Key: org.apache.tomcat** — **Language: java**, plus an **Alerts feed** link. The analysed system is **Apache Tomcat** — a real, mainstream, heavily-used Java codebase, not a strawman (BeyondTechnicalDebt p.7).

The argument the inventory makes. Itemising the dashboard exhaustively is the fastest way to internalise the deck's critique: here are *fourteen* panels of exact measurements about a serious production codebase — and not one of them ranks anything by what it *costs the team*. The violations panel says 8,794 Major items but cannot say which ten of them sit in code anyone will touch this quarter. The debt panel prices the total at 683 man-days but offers no first work item. The SIG spider chart grades four maintainability qualities but grades the *whole system*, not the files where the grades bite. Each panel is a *snapshot* statistic over the code product; what is missing — the slide's title asks "Actionable?" and slide 6 asks "What is missing?" — is the time dimension and the people dimension that turn measurement into priority (BeyondTechnicalDebt p.6–7, p.10). The exam-ready formula: *the dashboard measures everything about the code and nothing about how the organisation works with the code*.

Why Tomcat matters as the specimen. That the dashboard's project key reads `org.apache.tomcat` sharpens the point: this is not a toy or a failing project, it is one of the most widely deployed Java servers in existence, and *even here* a conventional tool can only report an undifferentiated 11.0% / 683-man-day debt pile. If the snapshot were going to be actionable anywhere, it would be on a codebase this mature and well-known — and it still is not (BeyondTechnicalDebt p.7–8).

The CodeScene pipeline, stage by stage (p.11)

The diagram, bottom to top. Page 11 is a layered architecture diagram of how behavioural analysis is computed (BeyondTechnicalDebt p.11). At the **bottom** sit the three inputs, each drawn with its icon: **Source Code** (a stack of code listings), **Version-Control Data** (the git logo), and **Project Management Tools, e.g. JIRA** (a board of ticket cards). Arrows feed all three upward into the first processing layer, a yellow band labelled "**Code, Process, and Evolutionary Metrics**" — the measurement layer, where the three metric families are computed (code metrics such as size and complexity from the source; process metrics such as change frequency and authorship from git; evolutionary metrics tracking how both develop over time). That layer feeds a red band labelled "**Pattern Detectors, Machine Learning and Intelligence**" — the interpretation layer, where raw metrics are turned into recognised patterns and predictions. The **top** box, labelled "**Visualizations, priorities, and predictive analytics**", shows three sample outputs side by side: a **prioritised findings table** (rows scored with red/orange severity cells) next to a **bar chart**, a **hotspot enclosure map**, and a **chord / edge-bundling diagram** (BeyondTechnicalDebt p.11).

Mapping the outputs to the lecture's concepts. The three output thumbnails are previews of the rest of the deck: the enclosure map is the hotspot visualisation demonstrated on ReactJS (p.13) and `checker.ts` (p.15); the chord diagram is the change-coupling X-Ray view (p.19); and the prioritised table with severity scores is the ranked, actionable counterpart to Sonar's flat violations list (p.7) — the same genre of output, but ordered by what matters. The pipeline diagram is therefore the lecture in one picture: *inputs that include history and people (git, JIRA), metrics that include process and evolution, intelligence that prioritises, outputs you can act on* (BeyondTechnicalDebt p.7, p.11, p.13, p.19).

Why JIRA is in the picture. The third input is easy to overlook but examinable: project-management data (e.g. JIRA tickets) lets the analysis link commits to *issue types* — which is how a tool can report a figure like `checker.ts`'s "**1,051 defects (133% bug fixes)**" (p.15): defect counts come from correlating the change history with the tickets that motivated the changes. Source code alone gives principal; git alone gives interest; the ticket system adds *why* each change happened (BeyondTechnicalDebt p.11, p.15).

Management / repayment strategies

What it is. This is the deck's *prescriptive* content — its actual recommendations for *how to manage and repay* debt. Notably it is a **prioritisation method**, not a refactoring catalogue (the catalogue of *how* to refactor is LO4's job; this lecture decides *which* and *when*). The strategy, assembled from across the deck, is:

1. **Don't try to fix everything** — the Tower-of-Babel slide (p.8) frames blanket repayment as hopeless; with "thousands of years" of accumulated debt you cannot repay it all, so the first and most important decision is *choosing a starting point*.
2. **Prioritise by hotspot = complexity × change frequency** (p.12) — repay the *high-interest* principal first; complex code you never touch can wait because it costs you nothing today.
3. **Validate with fault history** — direct effort where change history predicts the most faults (Graves et al., p.14), so the same work that reduces maintenance cost also reduces bugs.
4. **Drill to the function level with X-Ray** (p.16–18) — make the refactoring target *precise* (the two worst functions, not the whole file) so the fix is scoped, estimable, and affordable. This is exactly what makes a hotspot a tractable **Prefactoring** task instead of an open-ended rewrite.
5. **Watch trends, not just snapshots** (p.15) — act on rising complexity-vs-flat-LOC divergence *before* a file becomes unmanageable, rather than waiting until it is already the worst file in the system.
6. **Manage the social debt** — reduce knowledge loss and off-boarding risk (pair, document, redistribute ownership) using off-boarding simulation (p.22–23), because knowledge cannot be recovered once a person leaves.

7. **Let data support, not replace, judgement** — the concluding rule (p.25): rely on **human expertise**, and use data to **support developers' judgement and experience to get the highest ROI**.

What it's used for / why it matters. These strategies exist to convert the diagnostic machinery of the lecture (hotspots, trends, X-Ray, ownership maps) into an *actionable, ROI-ordered plan*. The unifying principle is **maximise return on a fixed repayment budget**: because you can never repay all debt, every unit of refactoring effort should go where it buys back the most future velocity, the most avoided bugs, and the most reduced risk. Each step narrows the focus — from "the whole codebase" to "the hotspots" to "the worst functions in the worst hotspot" — so that a limited budget lands precisely on the highest-value work.

When & how it's applied. A team with, say, two weeks of refactoring capacity per quarter applies these steps in order: run the behavioural analysis, take the top hotspots, confirm via fault history that they are also bug sources, X-Ray the top one to its two worst functions, check its complexity trend to confirm the debt is compounding, check ownership so they don't blindly refactor a single person's poorly-understood domain (or schedule knowledge transfer first), then spend their two weeks decomposing exactly those functions — and finally apply their own engineering judgement rather than mechanically "fixing whatever is reddest." That is the full management loop the deck prescribes.

Change-process link: This is the prioritisation layer over Lo4 refactoring. Once a hotspot is chosen, the actual repayment is ordinary **Prefactoring/Postfactoring** — applying Fowler's refactorings ([Fowler99]) and the maintainable-code / Clean Code guidelines ([Martin]) to drive down complexity (the principal). Behavioural analysis decides *which* code; refactoring is *how* you fix it.

"Beyond technical debt" — the deck's own framing

What it is. The title — *Beyond Technical Debt* (BeyondTechnicalDebt p.1) — is itself a thesis statement: it signals that the lecture deliberately goes *past* the naive, common understanding of technical debt (a single number from a static tool) in three specific directions:

1. **Beyond the snapshot** — from a static, point-in-time debt figure to an evolutionary *movie* that adds the time and people dimensions (p.9–11). The naive view measures the code once; the "beyond" view watches it evolve.
2. **Beyond the code** — debt is also **social**: legacy code, knowledge loss, off-boarding risk, "the technical debt that wasn't" (p.20–23). The naive view sees only code quality; the "beyond" view sees that people and knowledge are part of the debt. "*Technical debt isn't just technical*" (p.20).
3. **Beyond the tool** — data does not decide for you; "*ultimately, you need to rely on human expertise*" and use data to support judgement for highest ROI (p.25). The naive view trusts the metric; the "beyond" view treats the metric as decision *support*.

What it's used for / why it matters. This framing is the deck's *organising spine* — it tells you what the lecture is arguing *against* (debt as a static number you can read off a dashboard) and *for* (debt as an evolving, social, judgement-requiring phenomenon). Naming the three "beyonds" gives you a compact mental model of the whole lecture and a ready structure for an exam essay: each "beyond" corresponds to one of the deck's three movements (the snapshot-vs-movie methodology, the legacy/social section, and the human-expertise conclusion). It matters because each direction corrects a specific, common *mistake* practitioners make — over-trusting the snapshot, ignoring the people dimension, and letting the tool make the call.

When & how it's applied. Used as a checklist when reasoning about a real codebase's debt: Have I gone *beyond the snapshot* (am I looking at history and churn, not just a current debt figure)? Have I gone *beyond*

the code (have I checked ownership and off-boarding risk, not just complexity)? And am I staying *beyond the tool* (am I using the metrics to inform my judgement rather than mechanically obeying them)? The deck also notes that the methodology has more breadth than the lecture covers: the "There's much more in CodeScene" slide (p.24) lists further evolutionary, history-mined capabilities — **change coupling, microservices analysis** (with sub-points **shotgun surgery, team conflicts, technical sprawl**), **proactive warnings, retrospectives, delivery performance, and branch analyses** — all "beyond" what conventional static analysis can offer.

"There's much more in CodeScene" — the complete capability list (p.24)

The list, verbatim and in structure. The breadth slide (BeyondTechnicalDebt p.24, headline "*There's much more in CodeScene*") enumerates seven further capabilities, with three of them nested under Microservices exactly as follows:

- **Change coupling**
- **Microservices**
- **Shotgun surgery**
- **Team conflicts**
- **Technical sprawl**
- **Proactive warnings**
- **Retrospectives**
- **Delivery Performance**
- **Branch Analyses**

(BeyondTechnicalDebt p.24.) The slide gives the names only — no elaboration — so the safe exam treatment is to reproduce the list faithfully and gloss it cautiously. Grounded glosses: *Change coupling* is the temporal co-change analysis already demonstrated on p.19, here flagged as a general capability beyond the single X-Ray view. The *Microservices* trio names the failure modes evolutionary analysis can detect across service boundaries: *shotgun surgery* (one logical change forcing edits across multiple services — the same smell as p.19 but at architecture scale), *team conflicts* (multiple teams churning the same code — an organisation-and-people signal), and *technical sprawl* (proliferation across services). *Proactive warnings, retrospectives, delivery performance, and branch analyses* extend the same history-mining approach to early alerts, sprint reviews, throughput measurement, and branching behaviour respectively — all derived, like everything else in the deck, from the version-control record (BeyondTechnicalDebt p.24–25). The structural fact to memorise: **only Microservices has sub-bullets, and they are exactly three: shotgun surgery, team conflicts, technical sprawl.**

The four concluding theses, verbatim (p.25)

The slide's exact bullets. The conclusion slide ("To Conclude...", BeyondTechnicalDebt p.25) carries precisely four bullets, quoted here in full because exam answers benefit from reproducing them near-verbatim:

1. "*Technical debt is a real problem regardless of programming language*"
2. "*There's a huge amount of useful information stored in your version control system*"
3. "*Ultimately, you need to rely on human expertise*"
4. "*Support your developer's judgment and experience with data to get the highest ROI*"

Unpacking each thesis. (1) **Language-independence:** the deck's examples deliberately span Java (Tomcat, p.7), JavaScript (React, p.13, p.17–19, p.22–23), and TypeScript (`checker.ts`, p.15) — debt is a

property of evolving systems, not of any language's weaknesses. (2) **The VCS as a data asset**: every analysis in the deck — change frequency, trends, coupling, authorship, knowledge loss, off-boarding simulation — is mined from data the team *already has* in git; no new instrumentation is required, the information is lying in the repository waiting to be read (p.11). (3) **Human expertise is the final authority**: the metrics rank and reveal, but they do not decide — a red circle is a question, not an order. (4) **Data supports judgement for highest ROI**: the correct relationship is data *supporting* the developer's judgment and experience, with **ROI** as the optimisation target — the deck's last word frames debt repayment as an investment decision made by informed humans (BeyondTechnicalDebt p.25). Note how theses 3 and 4 together pre-empt the obvious misreading of the whole lecture ("just do whatever CodeScene says"): the tool's role is decision *support*, never decision *making*.

Resources slide (p.26)

The closing slide (BeyondTechnicalDebt p.26, "Resources") lists exactly two links, both pointing at the work of Adam Tornhill, the creator of CodeScene and author of *Your Code as a Crime Scene*:

<https://www.adamtornhill.com> (his main site) and **Tools**:

<https://www.adamtornhill.com/code/crimescenetools.htm> (the page collecting his behavioural code-analysis tooling). For follow-up study, these are the deck's sanctioned starting points for everything demonstrated in slides 9–24 (BeyondTechnicalDebt p.26).

The deck's visual rhetoric — slide imagery as memory anchors

The deck is unusually image-driven, and the images are chosen as arguments, not decoration. Cataloguing them gives you a set of retrieval cues that reliably bring back each slide's claim under exam pressure:

- **p.1 — the tortoise** crossing a vast gravel road under a grey sky, with "Beyond Technical Debt" overlaid: the system burdened by debt crawls; the road (the product's remaining life) is long. Anchor for the whole lecture's velocity theme (BeyondTechnicalDebt p.1).
- **p.3 — DNA double-helix backdrop** behind "Lehman's 'Laws' of Software Evolution": evolution imagery for evolution laws — change is written into a living system's nature (BeyondTechnicalDebt p.3).
- **p.4 — the Easy-vs-Simple speed chart**: blue (easy) decays, green (simple) endures — the loan drawn as a curve (BeyondTechnicalDebt p.4).
- **p.5 — Sisyphus, the BILL, and the beetles**: the business forever pushing the roadmap boulder; the central figure holding the invoice; the bug swarm reaching the users. One bill, two payees (BeyondTechnicalDebt p.5).
- **p.6 — a flat-lay wall of hand tools** (screwdrivers, pliers, wrenches, calipers, a spirit level) behind "Conventional Tools — What is missing?": a complete-looking toolkit that is nonetheless missing the one instrument that matters — something that measures *time and people* (BeyondTechnicalDebt p.6).
- **p.8 — Bruegel's Tower of Babel painting** under "Thousands of years of technical debt": a colossal, half-ruined, perpetually-under-construction edifice — accumulated complexity on a civilisational scale, with the caption-question "Where do you start when you want to pay it back?" (BeyondTechnicalDebt p.8).
- **p.9 — a dark title card** introducing CodeScene with the deck's pivotal metaphor: "*It's a 'movie' rather than a 'snapshot'...*" (BeyondTechnicalDebt p.9).
- **p.16 — a shoulder X-ray radiograph** introducing the X-Ray deep dive: seeing *inside* the body (the file) to find what is broken (BeyondTechnicalDebt p.16).
- **p.20 — a blurred optician's eye chart** behind "Legacy Code — Why technical debt isn't just technical": legibility degrading with distance — code whose meaning blurs as its authors recede (BeyondTechnicalDebt p.20).

Each pairing (image → claim) is deliberately redundant with the textual content above; in a semantic-search corpus and in an exam, redundancy across cues is a feature.

Link to maintainability & refactoring

What it is. This concept positions Lecture 11 within the wider course as the *measurement and prioritisation* counterpart to Lo4 (Refactoring & Maintainable Code) — it explains how technical debt, maintainability, hotspots, refactoring, and code smells all relate to one another:

- **Maintainability** is the quality *attribute* (how easy the system is to change); **technical debt** is the *deficit* in that attribute (how far short of ideal the code falls); and a **hotspot** is debt that is *actively costing you* (high interest) rather than dormant. So debt is "negative maintainability," and a hotspot is "the debt that hurts most right now."
- **Refactoring** ([Fowler99]) is the *repayment mechanism* — behaviour-preserving transformations that lower complexity (the principal) without changing what the code does.
- **Code smells** (e.g. Shotgun Surgery, p.19/p.24) are the *qualitative* indicators of debt (a developer's eye spotting a problem), while behavioural metrics (hotspots, change coupling, trends) are the *quantitative, prioritised, history-grounded* version of the same intuition.

What it's used for / why it matters. This linkage is what makes the lecture cohere with the rest of the course rather than standing alone. It clarifies a division of labour that is a likely exam point: **Lo4 tells you how to refactor; L11 tells you which debt to repay first and when.** Without L11's prioritisation, Lo4's refactoring techniques would be applied blindly across the codebase, spreading effort thinly; without Lo4's techniques, L11's hotspot ranking would identify targets you have no method to fix. Together they form a complete loop: *find the costly debt (behavioural analysis) → repay it (refactoring) → during the bracketing Prefactoring/Postfactoring phases of the change process.*

When & how it's applied. In the Rajlich process, **Prefactoring** and **Postfactoring** are *where* repayment happens, and this lecture tells you *which* debt to repay there *first* — so that the bracketing refactoring is aimed at the code where the next **Actualization** will otherwise be most expensive and most defect-prone. It also feeds two other phases: **Impact Analysis** (change coupling reveals hidden members of the impact set that a static dependency graph misses) and **Verification** (hotspots are where regressions are most likely, per Graves et al., so they should be tested hardest). Concretely, before changing a hotspot you Prefactor its two worst functions (Lo4 techniques) to lower the principal, you expand the impact set using its change-coupling partners, and you point your strongest tests at it during Verification.

JHotDraw Connection

The `BeyondTechnicalDebt` deck does **not** use JHotDraw — its worked examples are mined from large open-source repositories (ReactJS, p.13; TypeScript's `checker.ts`, p.15; a React `attach.*` module under X-Ray, p.18; React off-boarding simulation, p.22–23). So the JHotDraw connection here is *conceptual transfer*, made by this guide for exam purposes:

- **JHotDraw as a low-debt baseline.** JHotDraw is the course's exemplar of *clean, pattern-rich, maintainable* code (`00-overview.md`): it was built as a design exercise showcasing GoF patterns (Composite, Observer, Strategy, Factory). In debt terms, it is a system whose **principal** (complexity) was kept low by design — the opposite of the red-hot `checker.ts` hotspot. It is the "what good looks like" against which a CodeScene analysis would show mostly cool, low-frequency, low-complexity files. *What this is for:* it gives you a mental reference point — when reading a hotspot map, the pale, cool, rarely-changed circles are "JHotDraw-like" (healthy), and the dark-red, complex, churned circles are the debt.

- **Applying the hotspot lens to JHotDraw.** If you ran CodeScene over JHotDraw's git history, you'd expect any hotspots to cluster in its most-changed, most-complex areas (e.g. the `Figure` /drawing-editor hierarchy or command/undo machinery) — exactly the classes you'd target for **Prefactoring** before a feature change. The same **principal × interest** rule applies regardless of how clean the codebase started out: even a well-designed system develops local hotspots wherever change concentrates. *When applied:* before adding a feature to JHotDraw, you would compute its hotspots to predict which clean-looking classes are nonetheless expensive to touch because they change constantly.
- **Connecting to the course's JHotDraw change exercises.** When you perform **concept location** and **impact analysis** on JHotDraw (Lo2–Lo3), the change-coupling idea (p.19) tells you which JHotDraw classes historically co-change — a behavioural complement to the static dependency-graph search used in those phases. A located concept that lands in a (hypothetical) JHotDraw hotspot would warrant extra refactoring care, extra impact-set scrutiny, and harder verification.
- **Why JHotDraw is the right contrast.** Because JHotDraw deliberately minimised debt through good OO design and patterns, it illustrates the *preventive* half of the debt story (avoid borrowing in the first place) while this lecture covers the *diagnostic/repayment* half (find and pay back debt that already exists). Together they bracket the maintainability theme of the whole course: design it clean (JHotDraw, prevention) and, since Lehman guarantees debt will still accrue, measure and repay the worst of it (this lecture, cure).

Worked Example / Process Walkthrough

The deck embeds an end-to-end "find and prioritise the worst debt" walkthrough using the React/TypeScript codebases. Reconstructed as a process you could reproduce, and mapped onto the change process:

Scenario: You inherit a large codebase. There is "*thousands of years of technical debt*" and you must decide *where to start paying it back* (BeyondTechnicalDebt p.8). A conventional Sonar report says **11.0% / 683 man days / 10,072 violations** (p.7) — true but useless for prioritisation, because it tells you the *amount* of debt without telling you *which* debt is actually costing you.

1. **Reject the snapshot; get the movie.** Recognise that the static report tells you the *amount* of debt, not *where it matters*. Feed **source code + git history + JIRA** into a behavioural analysis (CodeScene) to add the **time** and **people** dimensions (p.9–11). *Why:* only the history can reveal which of those 10,072 violations sit in code you actually work with.
2. **Map hotspots.** Render the codebase as a nested-circle map (ReactJS, p.13): circle **size = complexity (principal)**, **redness = change frequency (interest rate)**. The dark-red inner circles (e.g. in `react-reconciler`, `react-devtools-shared`) are the hotspots — *complicated code you work with often* (p.12). **Start here**, not with the 10,072-item violation list. *How read:* ignore the big pale circles (complex but dormant); target the small-to-medium dark-red ones (the high-interest debt).
3. **Validate the choice with fault history.** Confirm the heuristic is sound: change-history measures predict faults better than size, and recent churn weighs most (Graves et al., p.14). The hotspots are therefore also your most likely **bug sources** — so fixing them yields double ROI (less maintenance cost and fewer defects).
4. **Read the trend, not just the dot.** Open the top hotspot's history (TypeScript `checker.ts`, p.15): **785 commits, 1,051 defects (133% bug fixes), Code Health 1**, and — decisively — a **complexity curve rising far faster than LOC**. The debt is *compounding*, not stable: act now, before it crosses into un-refactorable territory.
5. **Drill to functions with X-Ray.** The file is 25k LOC — too big to "just refactor". X-Ray ranks its functions (the React `attach.*` table, p.16–18). Target the worst: `updateFiberRecursively` (cyclomatic

complexity **48**, 220 LOC) and `inspectElementRaw` (**45**, 180 LOC). Now the refactoring is *scoped* to two functions — a bounded, estimable Prefactoring task rather than a file rewrite.

6. **Check change coupling.** X-Ray's chord diagram (p.19) shows which files change in lockstep with this one (e.g. `renderer.js` ↔ `attach`). Those co-changing files belong in the same repayment batch — and would belong in the **impact set** of any change here (which a static call graph would have missed). Fixing the coupling addresses *Shotgun Surgery*.
7. **Account for the social debt.** Check ownership/knowledge (p.20–23): is this hotspot a single author's domain? Simulate that author off-boarding (p.22→23) — if the region turns red, the debt is also an **off-boarding risk**; document/pair *before* fixing so the knowledge isn't lost mid-refactor.
8. **Repay via refactoring, guided by judgement.** Apply ordinary behaviour-preserving refactorings ([Fowler99], [Martin]) to drive down the two functions' complexity — this is **Prefactoring** if it precedes a feature, **Postfactoring** if it follows one. Per the conclusion (p.25): let the data *support* the developer's judgement for the **highest ROI**, don't refactor blindly just because a number is red.

Mapping to Rajlich's phases (drawn by this guide): Steps 1–6 are a *behavioural enrichment* of **Concept Location** and **Impact Analysis** — they tell you which code is expensive and which files co-change (so what's truly in the impact set). Steps 5–8 scope and execute **Prefactoring/Postfactoring** (the actual debt repayment). Throughout, the hotspot map tells you where **Verification** must be most thorough, because change-history predicts faults concentrate there (p.14).

A second walkthrough — the social-debt management loop (p.17, p.20–23)

The deck embeds a *second*, shorter end-to-end process for the people half of debt, runnable independently of the hotspot loop:

1. **Detect concentration.** While inspecting a hotspot's dashboard, read the **Main Author** field. The React DevTools backend shows **Brian Vaughn (80%)** — four-fifths of a 2,444-LOC hotspot in one head (BeyondTechnicalDebt p.17). Concentration this extreme means the file is "legacy in waiting" under the p.21 definition: the moment Vaughn leaves, it becomes code "we didn't write ourselves" for everyone remaining (p.21).
2. **Check current loss.** The same dashboard's **Knowledge Loss** field (6% abandoned on the backend file; 0% on `checker.ts`) tells you how much of the surrounding code has *already* crossed into abandonment — the Current Loss state of the simulation legend (p.15, p.17, p.22).
3. **Simulate the departure.** Open the off-boarding view ("How quickly can you turn your codebase into legacy code?"), scope to the directory (`react`), filter and select the developer ("brian" → Brian Vaughn), and re-render (p.22).
4. **Read the delta.** Compare before (map nearly uniform blue Knowledge) with "After they leave ..." (red Simulated Loss / Off-Boarding Risk regions across the packages) — the difference is the quantified bus-factor exposure for that one person (p.22–23).
5. **Mitigate while you still can.** Because knowledge loss is irreversible after the fact, every intervention must precede the departure: pair on the red regions, write down rationale, route other developers' changes through that code to broaden the authorship history. Done as part of an actual off-boarding plan, the red map is the prioritised handover checklist (p.20–23).
6. **Re-simulate.** After mitigation, the same simulation re-run shows shrunk red regions — the social analogue of a falling complexity trend (p.15, p.22–23).

The loop's exam value is its symmetry with the technical loop: hotspots are found by *complexity × churn* and repaid by refactoring; knowledge risk is found by *authorship × departure scenarios* and repaid by transfer — and both are computed from the same version-control history (p.11, p.25).

Slide-by-Slide Walkthrough (All 26 Pages)

This walkthrough narrates the deck in presentation order — what each slide shows, what it claims, and the one thing to retain — so that any fragment of a slide you half-remember can be re-anchored to its argument. It complements the Source Map table (which is optimised for lookup) with connected prose (optimised for understanding the flow).

Pages 1–2 — Title and definition

p.1 opens with the tortoise on the long gravel road, title "Beyond Technical Debt": the promise to go past the conventional treatment of debt. **p.2** delivers the working definition under the heading "Technical Debt": *"Stuff that isn't supposed to be there and is in the way of the stuff that is supposed to be there"*, attributed on the slide to *Building Evolutionary Architectures* (p. 110) by Neal Ford, Rebecca Parsons, and Patrick Kua (the slide prints the third author as "Patrick Kia" — a typo for Kua). The definition's two clauses split the work: "isn't supposed to be there" identifies the substandard material; "is in the way" restricts *debt* to the subset that obstructs current work — the restriction that the whole hotspot apparatus later operationalises (BeyondTechnicalDebt p.1–2, p.12).

Pages 3–5 — Why debt matters: Lehman, Hickey, business impact

p.3 ("Lehman", DNA-helix backdrop) quotes two of Lehman's "Laws" of Software Evolution: **Continuing Change** — *"a system must be continually adapted or it becomes progressively less satisfactory"* — and **Increasing Complexity** — *"as a system evolves, its complexity increases unless work is done to maintain or reduce it."* Together: change is mandatory, complexity is its default by-product, therefore debt accrues structurally. **p.4** ("Complexity Kills Development Speed") gives Rich Hickey's *Simple Made Easy* warning — ignore complexity and you *will* slow down; it *will* eventually kill you, "in a way that will make every sprint accomplish less" — beside the Easy(blue)-vs-Simple(green) speed-over-time chart whose curves cross. **p.5** ("Technical Debt Has Impact on Business & Product") routes the cost outward: the central BILL figure pays "Technical Debt" leftward to the **Roadmap** (Sisyphus painting; *what the business see: Long Lead Times, Lack of Predictability*) and rightward to the **Product** (beetle swarm; *what the users experience: bugs*). Retain: three slides, three registers — empirical law (p.3), practitioner economics (p.4), business consequence (p.5) (BeyondTechnicalDebt p.3–5).

Pages 6–8 — The critique of conventional tools

p.6 ("Conventional Tools — What is missing?", over the flat-lay of hand tools) poses the section's question. **p.7** ("Actionable?") answers it by exhibit: the complete Sonar dashboard of **Apache Tomcat** (key `org.apache.tomcat`, language java) — 162,306 LOC across 1,060 files; 1,447 classes in 103 packages with 14,271 methods; 26.6% comments; 7.1% duplications (22,906 lines, 566 blocks, 174 files — and the one live alert, "Duplicated lines (%) > 5"); complexity 3.1/method, 30.9/class, 42.2/file (total 44,773); rules compliance 83.7% with the SIG Maintainability Model spider chart (Analysability, Changeability, Stability, Testability); **10,072 violations** (0 Blocker, 0 Critical, **8,794 Major**, 65 Minor, 1,213 Info); 356 tags (all optional); an events history (Version 6.x 2010-07-26; Version 6.0.x 2009-06-07; Alert Orange 2009-02-15); and the circled headline **Technical Debt: 11.0% / \$ 341,563 / 683 man days** with its Duplication/Violations/Complexity/Comments breakdown pie. Every figure exact; none actionable. **p.8** ("Thousands of years of technical debt", Bruegel's Tower of Babel) draws the consequence as a question: *"Where do you start when you want to pay it back?"* Retain: precision ≠ actionability; the snapshot cannot rank (BeyondTechnicalDebt p.6–8).

Pages 9–11 — The CodeScene turn

p.9 (dark title card) introduces **Codescene** with the pivotal metaphor: *"It's a 'movie' rather than a 'snapshot'..."*. **p.10** states the thesis in prose: *"... static analysis will never be able to tell you if that excess code complexity actually matters – just because a piece of code is complex doesn't mean it's a problem. CodeScene identifies and prioritizes technical debt based on how the organization works with the code"* — sourced on the slide to *"How CodeScene Differs From Traditional Code Analysis Tools"* — and names the two added dimensions in italics: + **Time aspect** and + **Organization & people**. **p.11** shows the pipeline: Source Code + Version-Control Data (git) + Project Management Tools (e.g. JIRA) → *Code, Process, and Evolutionary Metrics* (yellow layer) → *Pattern Detectors, Machine Learning and Intelligence* (red layer) → *Visualizations, priorities, and predictive analytics* (top box, previewing a scored findings table, a hotspot map, and a chord diagram). Retain: the two missing dimensions, and the three-input/two-layer/one-output pipeline (BeyondTechnicalDebt p.9–11).

Pages 12–14 — Hotspots and their empirical basis

p.12 ("Hotspot") defines the central concept — *"A hotspot is a complicated code that you have to work with often"* — over the two-row diagram: **Principal** → circles growing along *Code Complexity*; **Interest Rate** → circles reddening along *Code Change Frequency*; both arrows converging on the dark-red **Hotspot** circle (with Android's `ActivityManagerService.java` legible in the faded background map). **p.13** ("ReactJS") shows the concept at scale: the React repository as an enclosure map — packages `react-dom`, `react-reconciler`, `react-devtools-shared`, `react`, `shared`, `scheduler`, `legacy-events`, `react-native-renderer`, `react-is`, `react-refresh`, `react-debug-tools`, `react-interactions`, `eslint-plugin-react-hooks` — with the dark-red hotspots clustered in `react-dom`, `react-reconciler`, and `react-devtools-shared`. **p.14** ("Hotspots are also common sources of bugs") grounds the heuristic in Graves, Karr, Marron & Siy (IEEE TSE 26(7):653–661, August 2000): process measures from change history beat product metrics; number of changes beats length; a year-older module has roughly a third fewer faults; the best model sums all past changes with large, recent ones weighted most. Retain: definition (p.12), visualisation (p.13), evidence (p.14) (BeyondTechnicalDebt p.12–14).

Page 15 — Worrisome trends

p.15 ("Worrisome Trends") is the deck's anatomy of one pathological file: `TypeScript/src/compiler/checker.ts` — Size 25,774 LOC; Code Health **1**; Change Frequency 785 commits; Main Author Anders Hejlsberg (26%); Knowledge Loss 0% abandoned; Defects 1,051 (133% bug fixes); shown as a giant dark-red circle beside tiny neighbours (`commandLineParser.ts`, `transform...`), with the Complexity Trend chart (October 2015 → April 2019) plotting the red complexity line climbing to ~130,000 over a nearly flat blue LOC line and a flatter green code-comments line, captioned "Click on a point to diff the code changes". Retain: the **divergence** (complexity up, LOC flat) is the warning signature; the file is technically dire yet socially healthy (0% knowledge loss) (BeyondTechnicalDebt p.15).

Pages 16–18 — X-Ray

p.16 (shoulder radiograph) titles the deep dive: **X-Ray**. **p.17** sets the target: `react/packages/react-devtools-shared/src/backend` — 2,444 LOC, amber Code Health, 187 commits, Main Author **Brian Vaughn (80%)**, Knowledge Loss 6%, Defects 24 (20% bug fixes), last modified 6 months ago; action buttons View Code / X-Ray / Trends / Code Review; text: *"We've identified a problematic file but it's still huge! Let's X-Ray it to find the most problematic functions."* **p.18** delivers the function table (tabs: Structural Recommendations, Change Frequency Distribution) ranking `attach` functions by Change Frequency / LOC / Cyclomatic Complexity / Overloaded Functions?: `attach (top-level)` 103/109/9; `recordMount` 37/48/8;

handleCommitFiberRoot 32/89/19; flushPendingEvents 30/105/15; inspectElement 30/93/12; flushInitialOperations 29/51/6; updateFiberRecursively 24/220/48; inspectElementRaw 24/180/45; recordUnmount 18/50/9 — red shading on 19/15/48/45. Retain: the refactoring targets are the red-complexity, non-trivially-churned rows (updateFiberRecursively , inspectElementRaw), *not* the most-changed row (BeyondTechnicalDebt p.16–18).

Page 19 — Change coupling

p.19 ("Change Coupling") shows the X-Ray chord diagram centred on `renderer.js - attach` (top-level context), with thick blue ribbons to `CommitTreeBuilder.js - updateTree.switch` (magenta rim arc) and a second `attach` entity, and faded labels for `CommitTreeBuilder.js - updateTree` (top-level context), `SelectedElement.js - SelectedElement`, `renderer.js - attach.flushPendingEvents`, and an `agent.js` entity. Retain: ribbons = co-change mined from commits; cross-file lockstep = hidden impact-set members + Shotgun Surgery smell (BeyondTechnicalDebt p.19, p.24).

Pages 20–23 — Legacy code and off-boarding

p.20 (blurred eye chart) titles the social half: "Legacy Code — Why Technical debt isn't just technical". p.21 defines it: legacy code is typically used to describe code that **lacks in quality (relative perspective)** and that **we didn't write ourselves** (the second bullet bolded on the slide); below, "The Technical Debt That Wasn't" shows Product #1 and #2 with thumbs-up owners and **Product #3 with only a "?"** — the abandoned product whose debt is pure ownership. p.22 ("How quickly can you turn your codebase into legacy code?") presents the off-boarding simulator on the `react` directory: legend **Knowledge / Current Loss / Simulated Loss / Off-Boarding Risk / Inconclusive**; instruction *"Simulate the effects of a planned off-boarding if some developers leave your organization"*; developer filter showing "brian" → **Brian Vaughn**; map almost uniformly blue. p.23 ("After they leave ...") re-renders with *Simulated offboarded authors: Brian Vaughn*: red Simulated-Loss regions bloom across the packages. Retain: the definition's two bullets; the five legend states; and the continuity that Vaughn was the 80% main author from p.17 (BeyondTechnicalDebt p.17, p.20–23).

Pages 24–26 — Breadth, conclusions, resources

p.24 ("There's much more in CodeScene") lists: Change coupling; Microservices (→ Shotgun surgery, Team conflicts, Technical sprawl); Proactive warnings; Retrospectives; Delivery Performance; Branch Analyses. p.25 ("To Conclude...") states the four theses: debt is a real problem regardless of programming language; a huge amount of useful information is stored in your version control system; ultimately you need to rely on human expertise; support your developer's judgment and experience with data to get the highest ROI. p.26 ("Resources") closes with `adamtornhill.com` and `adamtornhill.com/code/crimescenetools.htm`. Retain: the conclusion's order — reality of the problem, richness of the VCS, primacy of human expertise, data as ROI-maximising support (BeyondTechnicalDebt p.24–26).

Compare & Contrast Tables

Dense, exam-oriented contrasts. Every cell is grounded in the deck pages cited in each table's caption.

Snapshot vs movie (static vs behavioural analysis)

Dimension	Conventional / static ("snapshot")	Behavioural / evolutionary ("movie")
Data source	Current source code only	Source code + version-control history + project-management data (JIRA)

Dimension	Conventional / static ("snapshot")	Behavioural / evolutionary ("movie")
Time dimension	None — one point in time	Central — trends, change frequency, recency
People dimension	None	Central — authorship, knowledge loss, off-boarding risk
Typical output	Violation counts, debt %, man-days (e.g. 10,072 violations; 11.0%; 683 man-days)	Prioritised hotspots, complexity trends, change coupling, knowledge maps
Question answered	"How much debt is there?"	"Which debt is costing us, and where do we start?"
Failure mode	Precise but not actionable — cannot say whether complexity <i>matters</i>	Requires history; outputs still need human judgement (p.25)
Deck exhibit	Tomcat Sonar dashboard (p.7)	CodeScene on React/TypeScript (p.13, p.15, p.17–19, p.22–23)

(BeyondTechnicalDebt p.6–11.)

The complexity × change-frequency quadrants

	Low change frequency	High change frequency
Low complexity	Healthy, quiet code — no debt concern; the pale small circles on the map	Active but simple code — cheap to keep changing; high interest on near-zero principal (e.g. <code>attach</code> top-level: 103 changes, complexity 9, p.18)
High complexity	Dormant debt — big pale circles; principal with no interest; <i>not</i> a priority (the "biggest/ugliest file" trap)	Hotspot — "complicated code that you have to work with often" (p.12); the dark-red circles; the rational first repayment target

(BeyondTechnicalDebt p.12–13, p.18.)

The deck's two worked file dashboards

Field	<code>checker.ts</code> (p.15)	<code>react-devtools-shared/src/backend</code> (p.17)
Codebase	TypeScript compiler	React DevTools
Size	25,774 LOC	2,444 LOC
Code Health	1 (red — worst band)	amber/yellow
Change Frequency	785 commits	187 commits
Main Author	Anders Hejlsberg (26%)	Brian Vaughn (80%)
Knowledge Loss	0% abandoned	6% abandoned
Defects	1,051 (133% bug fixes)	24 (20% bug fixes)
Last modified	~0 months ago	6 months ago
Dominant risk	Technical — runaway complexity trend, defect factory	Social — single-owner knowledge concentration (the p.22–23 simulation subject)
Role in deck	"Worrisome trends" specimen	X-Ray entry point

(BeyondTechnicalDebt p.15, p.17, p.22–23.)

Technical debt vs social (knowledge) debt

Aspect	Technical debt (code)	Social debt (legacy/knowledge)
Substance	Excess complexity in the code (principal)	Missing knowledge about the code (departed authors)
Detected by	Complexity metrics × change frequency (hotspots, p.12)	Authorship mining: knowledge loss, abandoned code, off-boarding simulation (p.21–23)
Can a clean file have it?	No — by definition it is a code property	Yes — "the technical debt that wasn't": fine code with no owner (p.21)
Repayment	Refactoring (Prefactoring/Postfactoring; [Fowler99])	Organisational: pairing, documentation, ownership redistribution — <i>before</i> departure
Reversible after the fact?	Yes — code can always be refactored later (at a price)	No — once the person leaves, the knowledge is gone; only advance mitigation works
Deck slogan	"Complicated code that you have to work with often" (p.12)	"Why technical debt isn't just technical" (p.20)

(BeyondTechnicalDebt p.12, p.20–23.)

Change coupling vs static dependency

Aspect	Static dependency	Change coupling
Derived from	Code structure: imports, calls, type references	Commit history: entities edited in the same commits
Visible to static analysis?	Yes	No — invisible without the VCS
Can exist without the other?	Yes (depended-on but never co-changed)	Yes (no structural link, yet always co-changed — the dangerous case)
Use in impact analysis	The conventional impact set	Evidence-based <i>correction</i> to the impact set (catches what statics miss)
Smell it reveals	Cyclic/dense coupling	Shotgun Surgery — one logical change smeared over files (p.19, p.24)
Deck exhibit	—	<code>renderer.js attach</code> ↔ <code>CommitTreeBuilder.js updateTree.switch</code> chord (p.19)

(BeyondTechnicalDebt p.19, p.24.)

L04 (Refactoring) vs L11 (Technical debt) — division of labour

Question	L04 answers	L11 answers
<i>How</i> do I improve this code?	Yes — the refactoring catalogue, behaviour-preserving transformations	No
<i>Which</i> code should I improve first?	No	Yes — hotspot ranking (complexity × change frequency)
<i>When</i> is the debt compounding?	No	Yes — complexity trends (p.15)
<i>Who</i> dimension?	No	Yes — knowledge loss, off-boarding (p.20–23)

Question	L04 answers	L11 answers
Where in the change process?	Executes inside Prefactoring/Postfactoring	Prioritises what Prefactoring/Postfactoring should target

(BeyondTechnicalDebt p.12, p.15, p.20–23; division drawn by this guide, consistent with 00-overview.md.)

Numbers to Memorise

Every load-bearing figure in the deck, in one table — these are the numbers an examiner can probe and the anchors semantic search should hit:

Figure	Value	Context	Page
Book page for the debt definition	p. 110	<i>Building Evolutionary Architectures</i> (Ford, Parsons, Kua)	p.2
Tomcat lines of code	162,306 (325,036 lines; 87,758 statements; 1,060 files)	Sonar dashboard	p.7
Tomcat classes	1,447 (103 packages; 14,271 methods; + 1,262 accessors)	Sonar dashboard	p.7
Tomcat comments	26.6% (58,991 lines; 59.1% docu. API; 5,418 undocu. API; 1,164 commented LOCs)	Sonar dashboard	p.7
Tomcat duplications	7.1% (22,906 lines; 566 blocks; 174 files)	the one tripping alert: "Duplicated lines (%) > 5"	p.7
Tomcat complexity	3.1/method; 30.9/class; 42.2/file; total 44,773	Sonar dashboard	p.7
Tomcat rules compliance	83.7%	with SIG model spider (A/C/S/T)	p.7
Tomcat violations	10,072 (Blocker 0; Critical 0; Major 8,794 ; Minor 65; Info 1,213)	Sonar dashboard	p.7
Tomcat technical debt	11.0% / \$ 341,563 / 683 man days	the "Actionable?" headline	p.7
Graves et al. publication	IEEE TSE 26(7):653–661, August 2000	fault-prediction study	p.14
Age effect on faults	~one-third fewer faults for a year-older module	Graves et al.	p.14
<code>checker.ts</code> size	25,774 LOC	TypeScript compiler	p.15
<code>checker.ts</code> change frequency	785 commits		p.15
<code>checker.ts</code> main author	Anders Hejlsberg, 26%		p.15
<code>checker.ts</code> knowledge loss	0% abandoned code		p.15
<code>checker.ts</code> defects	1,051 (133% bug fixes)		p.15
<code>checker.ts</code> Code Health	1 (worst band)		p.15

Figure	Value	Context	Page
<code>checker.ts</code> complexity trend	→ ~130,000 over Oct 2015–Apr 2019, LOC flat	the divergence signature	p.15
DevTools backend size	2,444 LOC	X-Ray target file	p.17
DevTools backend change frequency	187 commits		p.17
DevTools backend main author	Brian Vaughn, 80%	the later off-boarding subject	p.17, p.22–23
DevTools backend knowledge loss	6% abandoned		p.17
DevTools backend defects	24 (20% bug fixes)		p.17
Worst X-Ray functions	<code>updateFiberRecursively</code> CC 48 (220 LOC, 24 changes); <code>inspectElementRaw</code> CC 45 (180 LOC, 24 changes)	the refactoring targets	p.18
Most-changed X-Ray function	<code>attach (top-level context)</code> : 103 changes, CC only 9	the non-target	p.18
Other red-flagged functions	<code>handleCommitFiberRoot</code> CC 19; <code>flushPendingEvents</code> CC 15		p.18
Legend states (off-boarding)	5: Knowledge, Current Loss, Simulated Loss, Off-Boarding Risk, Inconclusive		p.22–23
"Much more" list	7 items, 3 sub-items under Microservices		p.24
Concluding theses	4 bullets		p.25

Definitions & Terminology

Each row below gives, in compressed form, *what* the term is, *what it is used for*, and (where useful) *how it is measured or applied*.

Term	Definition (as used in the deck)	Source
Technical debt	<i>What</i> : "Stuff that isn't supposed to be there and is in the way of the stuff that is supposed to be there" — substandard code that obstructs current work. <i>Used for</i> : framing code-quality deficits as a financial loan (borrow speed now, pay interest later) so the cost is negotiable with the business. <i>Applied</i> : identify it via the principal/interest split and prioritise repayment by hotspot.	BeyondTechnicalDebt p.2 (Ford/Parsons/Kua)
Principal	<i>What</i> : the substandard code itself — measured as code complexity (size, cyclomatic complexity, nesting), a static property. <i>Used for</i> : representing "how much you borrowed." <i>Applied</i> : high principal alone is <i>dormant</i> debt — it only costs when multiplied by interest, so it is never ranked on its own.	p.12
Interest rate	<i>What</i> : the recurring cost of the principal — measured as change frequency (how often the code is modified in the VCS), a behavioural property. <i>Used for</i> : representing "how fast the debt accrues cost." <i>Applied</i> : mined from the git log as commit-count per file; the missing dimension static tools cannot see.	p.12

Term	Definition (as used in the deck)	Source
Hotspot	<i>What:</i> "Complicated code that you have to work with often" = high complexity (principal) × high change frequency (interest). <i>Used for:</i> the rational <i>first</i> repayment target out of thousands of files — where interest is largest and future change costliest. <i>Applied:</i> computed by ranking files on complexity × commit-frequency; visualised as small dark-red circles on the enclosure map.	p.12
Lehman — Continuing Change	<i>What:</i> "A system must be continually adapted or it becomes progressively less satisfactory." <i>Used for:</i> proving debt is structural — a useful system <i>must</i> keep changing or its fitness decays. <i>Applied:</i> justifies a <i>standing</i> (recurring) refactoring budget rather than one-off cleanups.	p.3
Lehman — Increasing Complexity	<i>What:</i> "As a system evolves, its complexity increases unless work is done to maintain or reduce it." <i>Used for:</i> showing complexity creep is the <i>default</i> trajectory, not bad luck. <i>Applied:</i> justifies deliberate, separately-budgeted complexity-reduction effort aimed (via trends) at the files climbing fastest.	p.3
Snapshot (conventional analysis)	<i>What:</i> a static, point-in-time report of violations/debt (e.g. SonarQube's "11.0% / 683 man days"). <i>Used for:</i> telling you <i>how much</i> debt exists. <i>Limitation/applied:</i> precise but not actionable — it cannot say <i>where to start</i> or whether complexity <i>matters</i> , because it has no time/people data.	p.6–8
Movie (behavioral analysis)	<i>What:</i> an evolutionary view over the VCS history adding the time aspect and organization & people dimensions (CodeScene). <i>Used for:</i> answering "where do we start?" by ranking debt by how people actually work with the code. <i>Applied:</i> feed git history (+JIRA) through metrics + ML to get prioritised hotspots, trends, and ownership maps.	p.9–10
CodeScene	<i>What:</i> Adam Tornhill's behavioural code-analysis tool. <i>Used for:</i> identifying and prioritising technical debt based on how the organization works with the code (not just its static quality). <i>Applied:</i> produces hotspot maps, X-Ray drill-downs, complexity trends, change-coupling diagrams, and off-boarding simulations from VCS data.	p.9–11
Code/Process/Evolutionary metrics	<i>What:</i> the three families of measurements derived from source + version control + JIRA. <i>Used for:</i> feeding the pattern detectors/ML that produce priorities and predictions. <i>Applied:</i> code metrics = complexity (principal); process/evolutionary metrics = change frequency, churn, authorship (interest + social signals).	p.11
X-Ray	<i>What:</i> a CodeScene "deep dive" that drills <i>inside</i> a hotspot file to rank its functions by change frequency × cyclomatic complexity (and show change coupling). <i>Used for:</i> turning an un-actionable "this 25k-line file is a hotspot" into a precise, scoped fix. <i>Applied:</i> read the function table, refactor the two highest-complexity, frequently-changed functions first.	p.16–18
Change coupling	<i>What:</i> files/functions that historically change together in the same commits — a <i>temporal</i> dependency invisible to static call graphs. <i>Used for:</i> (a) detecting Shotgun Surgery (a debt smell), and (b) correcting the impact set in impact analysis. <i>Applied:</i> read the X-Ray chord diagram; entities tied by strong arcs belong together in the impact set and the repayment batch.	p.19, p.24
Complexity trend	<i>What:</i> the trajectory of a file's complexity over time (vs its LOC). <i>Used for:</i> distinguishing <i>managed</i> debt (flat/co-rising) from <i>compounding</i> debt (complexity rising while LOC is flat). <i>Applied:</i> open the top hotspot's trend chart; act when the red complexity curve outpaces the blue LOC curve.	p.15
Code Health	<i>What:</i> a CodeScene per-file quality score (low = unhealthy; checker.ts scored 1, the worst band). <i>Used for:</i> a quick single-number health flag to confirm a hotspot is genuinely in trouble. <i>Applied:</i> used alongside change frequency, defect ratio, and the trend to confirm a file needs intervention now.	p.15

Term	Definition (as used in the deck)	Source
Legacy code	<i>What:</i> code that (a) lacks quality (a <i>relative</i> , perspective-dependent judgement) and/or (b) we didn't write ourselves (the ownership/knowledge dimension). <i>Used for:</i> reframing debt as partly a <i>people</i> problem — code becomes legacy when its knowledge leaves, not only when it is bad. <i>Applied:</i> measured from VCS authorship; mitigated organisationally (document, pair, redistribute), not just by refactoring.	p.20–21
Knowledge loss / abandoned code	<i>What:</i> code whose authors are gone — measurable from the VCS (e.g. "0% abandoned code" on <code>checker.ts</code>). <i>Used for:</i> flagging code that is risky to change because nobody understands it, regardless of its intrinsic quality. <i>Applied:</i> the "Knowledge Loss" field; high values mark code where the next change will be slow and dangerous.	p.15, p.21
Off-boarding risk / simulation	<i>What:</i> a prediction of how much code would become legacy if specific developers left, by <i>simulating</i> their departure over the authorship map. <i>Used for:</i> exposing bus-factor risk <i>before</i> the knowledge walks out the door. <i>Applied:</i> select a developer, read the before/after map; red regions = act now (document/pair/redistribute) while that person is still present.	p.22–23
"The technical debt that wasn't"	<i>What:</i> abandoned/unowned code (the "?" product) whose risk is <i>organisational</i> rather than in the code itself. <i>Used for:</i> illustrating that debt can be a pure ownership problem — clean code with no owner is still a liability. <i>Applied:</i> find zero-owner regions in the VCS and assign ownership before they must be changed.	p.21
Shotgun surgery	<i>What:</i> a code smell where one logical change is spread across many files. <i>Used for:</i> naming the design defect that high change coupling reveals. <i>Applied:</i> spotted as strong arcs in the change-coupling diagram; repaid by consolidating the co-changing logic into a cohesive unit.	p.24 (and [Fowler99])

One-Paragraph Summaries (Rapid Revision)

Each paragraph compresses one movement of the deck into a recitable unit. These are deliberately redundant with the long-form sections above — use them for last-pass revision and as semantic-search anchors.

The problem (p.1–5). Technical debt is "stuff that isn't supposed to be there and is in the way of the stuff that is supposed to be there" (Ford/Parsons/Kua, p.2). It is structurally inevitable — Lehman: systems must continually change or decay, and evolving systems grow more complex unless work counteracts it (p.3). Its cost is velocity — Hickey: ignored complexity invariably slows you down and eventually "will make every sprint accomplish less" (p.4) — and the bill is paid outward twice: long lead times and lack of predictability to the business, bugs to the users (p.5).

The critique (p.6–8). Conventional static tools produce a precise snapshot — Tomcat under Sonar: 162,306 LOC, 10,072 violations (8,794 Major), Technical Debt 11.0% / \$ 341,563 / 683 man days, SIG maintainability spider, duplication alert — yet answer nothing about priority (p.7). With "thousands of years of technical debt" accumulated (Tower of Babel), the only question that matters is "where do you start when you want to pay it back?" — and a snapshot cannot say (p.8).

The method (p.9–14). CodeScene is "a 'movie' rather than a 'snapshot'" (p.9): it adds the time aspect and the organization-and-people dimension, prioritising debt "based on how the organization works with the code", because "static analysis will never be able to tell you if that excess code complexity actually matters" (p.10). Pipeline: source + git + JIRA → code/process/evolutionary metrics → pattern detectors/ML → visualizations, priorities, predictive analytics (p.11). Its core unit is the hotspot — "complicated code that you have to work with often": principal = complexity, interest rate = change frequency (p.12), visualised as size × redness on enclosure maps (ReactJS, p.13), and empirically validated by Graves et al. 2000: change-history measures out-predict product metrics; changes beat length; large recent changes weigh most (p.14).

The drill-down (p.15–19). Trends separate managed from compounding debt: `checker.ts` (25,774 LOC, 785 commits, 1,051 defects at 133% bug fixes, Code Health 1) shows complexity climbing to ~130,000 over flat LOC — rotting in place (p.15). X-Ray applies hotspot logic inside the file: the 2,444-LOC React DevTools backend (187 commits, Brian Vaughn 80%) decomposes into a function table where `updateFiberRecursively` (CC 48) and `inspectElementRaw` (CC 45) — not the most-churned `attach` top-level (103 changes, CC 9) — are the scoped targets (p.16–18). The chord diagram exposes change coupling — `renderer.js attach` ↔ `CommitTreeBuilder.js updateTree.switch` — temporal dependencies invisible to static analysis (p.19).

The social half (p.20–23). Technical debt isn't just technical: legacy code is code that lacks quality (relatively) and that *we didn't write ourselves* (p.20–21); "the technical debt that wasn't" is the unowned Product #3 (p.21). The off-boarding simulator answers "how quickly can you turn your codebase into legacy code?" — select Brian Vaughn, and react's blue Knowledge map erupts in red Simulated Loss / Off-Boarding Risk "after they leave ..." (p.22–23). Knowledge, unlike complexity, cannot be refactored back: mitigate before departure or not at all.

The bottom line (p.24–26). There's much more in CodeScene — change coupling; microservices (shotgun surgery, team conflicts, technical sprawl); proactive warnings; retrospectives; delivery performance; branch analyses (p.24) — but the conclusion is tool-humble: debt is real regardless of language; the VCS holds a huge amount of useful information; ultimately you rely on human expertise; and data's job is to support the developer's judgment and experience for the highest ROI (p.25). Further reading: adamtornhill.com (p.26).

Glossary of Proper Names (People, Tools, Systems)

Everyone and everything the deck names or shows, with their role in the argument:

- **Neal Ford, Rebecca Parsons, Patrick Kua** — authors of *Building Evolutionary Architectures*, the source (its p. 110) of the deck's working definition of technical debt. The slide itself prints the third author as "Patrick Kia" — a typo for Kua (BeyondTechnicalDebt p.2).
- **Manny Lehman** — originator of the "Laws" of Software Evolution; the deck quotes **Continuing Change** and **Increasing Complexity** (BeyondTechnicalDebt p.3).
- **Rich Hickey** — creator of Clojure; his talk *Simple Made Easy* supplies the "complexity will eventually kill you / every sprint accomplish less" argument and the Easy-vs-Simple speed chart (BeyondTechnicalDebt p.4).
- **SonarQube / Sonar** — the conventional static-analysis platform whose dashboard (from the austinjug.org "Heintz Technical Debt Sonar" presentation) is the deck's specimen of a precise-but-unactionable snapshot (BeyondTechnicalDebt p.7).
- **SIG Maintainability Model** — the maintainability-quality model on the Sonar dashboard's spider chart, with axes **(A)nalysability**, **(C)hangeability**, **(S)tability**, **(T)estability** (BeyondTechnicalDebt p.7).
- **Apache Tomcat** (`org.apache.tomcat`, java) — the real production system the Sonar dashboard analyses; chosen force of example: even a mainstream, mature codebase yields only an undifferentiated debt pile under static analysis (BeyondTechnicalDebt p.7).
- **Pieter Bruegel's Tower of Babel** — the painting on the "Thousands of years of technical debt" slide (identification of the painting by this guide); the image of accumulated, unfinishable construction debt (BeyondTechnicalDebt p.8).
- **CodeScene** — Adam Tornhill's behavioural code-analysis tool; the deck's vehicle for every movie-style analysis: hotspots, trends, X-Ray, change coupling, knowledge maps, off-boarding simulation (BeyondTechnicalDebt p.9–24).

- **git** — the version-control system whose history is the analysis's central input (logo shown in the pipeline) (BeyondTechnicalDebt p.11).
- **JIRA** — the example project-management input; ties commits to issue types, enabling defect statistics like "133% bug fixes" (BeyondTechnicalDebt p.11, p.15).
- **ActivityManagerService.java** — the Android platform class faintly visible behind the hotspot definition diagram; a canonical real-world hotspot (BeyondTechnicalDebt p.12).
- **ReactJS** — Facebook's UI library; its repository provides the hotspot map (p.13), the X-Ray case study (p.17–18), the change-coupling chord diagram (p.19), and the off-boarding simulation (p.22–23) (BeyondTechnicalDebt p.13, p.17–19, p.22–23).
- **Todd L. Graves, Alan F. Karr (RTI International), J. S. Marron, Harvey Siy (University of Nebraska at Omaha)** — authors of *Predicting Fault Incidence Using Software Change History*, IEEE TSE 26(7):653–661, August 2000 — the empirical backbone of hotspot prioritisation (BeyondTechnicalDebt p.14).
- **TypeScript / checker.ts** — Microsoft's typed-JavaScript compiler and its 25,774-LOC type checker, the "worrisome trends" specimen (BeyondTechnicalDebt p.15).
- **Anders Hejlsberg** — TypeScript's lead architect (also known for Turbo Pascal, Delphi, C#); main author (26%) of `checker.ts` on the slide (BeyondTechnicalDebt p.15).
- **Brian Vaughn** — React DevTools maintainer; main author (80%) of the X-Ray target file and the subject of the off-boarding simulation (BeyondTechnicalDebt p.17, p.22–23).
- **Adam Tornhill** — author of *Your Code as a Crime Scene* and creator of CodeScene; the Resources slide points to adamtornhill.com and his crime-scene tools page (BeyondTechnicalDebt p.26).

Common Pitfalls / Gotchas

- **Conflating principal with cost.** A complex file you never touch is *dormant* debt — it charges no interest. The cost is **principal × interest (change frequency)**, not complexity alone. This is the whole reason a global "11% technical debt" figure is misleading (BeyondTechnicalDebt p.7, p.12). *Why it bites*: it makes teams "fix the biggest/ugliest file" when the real cost is the moderately-complex file they edit every sprint.
- **Trusting the snapshot as "actionable".** The Sonar dashboard is precise (162,306 LOC, 10,072 violations, 683 man-days) but the slide literally titles it "*Actionable?*" — it doesn't tell you **where to start** (p.7–8). Don't mistake precision for usefulness: an exact number you can't act on is no better than a guess.
- **Believing static analysis can rank what matters.** "*Static analysis will never be able to tell you if that excess code complexity actually matters.*" (p.10). Complexity is necessary but not sufficient to flag a problem; you need the change-frequency dimension to separate the costly debt from the dormant debt.
- **Reading the hotspot map wrong.** In CodeScene's circle maps, **size = complexity** and **colour (red) = change frequency** — two *different* axes. A big pale circle (complex, rarely changed) is **not** a priority; a small dark-red circle (simple-ish but constantly churned) might be. Don't read size alone — read the *combination*, because the hotspot is the overlap of both.
- **Forgetting that change frequency beats size for faults.** The research is explicit: *number of times changed* predicts faults better than *length*, and **recent** changes weigh most (Graves et al., p.14). Don't prioritise the biggest file; prioritise the most-changed-*recently* one.
- **Acting on a single data point instead of the trend.** A file's current complexity matters less than its **trajectory**; rising complexity against flat LOC is the real alarm (p.15). The tool is a *movie* for a reason — a high but *stable* file may be fine, while a moderate but *climbing* file is the one about to become unmanageable.

- **Trying to refactor the whole hotspot file.** A 25k-LOC hotspot is not a refactoring task; **X-Ray to the two worst functions** and scope the fix (p.16–18). Boiling the ocean is the failure mode the Tower-of-Babel slide warns against (p.8) — bounded, function-level work is what actually gets done.
- **Treating debt as purely technical.** *"Technical debt isn't just technical"* (p.20). Knowledge loss and off-boarding risk are debt too — code becomes "legacy" when its authors leave, not only when it's bad (p.21–23). Ignoring the people dimension misses real, un-refactorable risk (you can't refactor knowledge back after someone has left).
- **Letting the tool make the decision.** The conclusion is emphatic (p.25): *"ultimately, you need to rely on human expertise."* Data **supports** judgement to get the **highest ROI**; it does not replace the developer. Blindly refactoring whatever is reddest is the wrong lesson — the metric points, the human decides.
- **Assuming "legacy = old."** The deck's definition is *relative* (lacks quality from a perspective) and *ownership-based* (didn't write it ourselves) — a brand-new file can be "legacy" if its author already left (p.21). Don't equate legacy with age; equate it with *missing knowledge and relative low quality*.
- **Missing hidden coupling in impact analysis.** Files with no static dependency can still always change together (change coupling, p.19). A static call-graph impact analysis will under-count the impact set; the VCS history corrects it — so trust the co-change evidence even when the dependency graph is silent.

Easily Confused Distinctions (Exam Traps)

Pairs and triples that look interchangeable but are graded as different things. For each: the distinction, and the one-line discriminator to write.

Complexity vs hotspot

Complexity is *one axis* (the principal); a hotspot is the *conjunction* of high complexity **and** high change frequency (BeyondTechnicalDebt p.12). Discriminator: *every hotspot is complex, but most complex code is not a hotspot — without churn it is dormant debt*.

Principal vs interest rate

Principal = the code complexity you took on (static, measurable from source alone); interest rate = the change frequency that makes the principal cost you (behavioural, measurable only from the VCS) (BeyondTechnicalDebt p.12). Discriminator: *principal is what you borrowed; interest is how often you're forced to pay on it*. Trap: calling change frequency the "principal" inverts the metaphor.

Easy vs simple (Hickey)

"Easy" = convenient right now; "simple" = un-entangled. The p.4 chart plots them as opposed speed curves: easy wins early, simple wins the long haul (BeyondTechnicalDebt p.4). Discriminator: *easy is about effort-to-adopt today; simple is about structure that sustains speed*.

Hickey vs Graves — argument vs evidence

Hickey's claim is explicitly *"based on experience"* — a practitioner's contention (p.4); Graves et al. is a peer-reviewed statistical study (IEEE TSE 2000, p.14). Discriminator: *cite Hickey for the motivating economics, Graves for the empirical validation*. Trap: presenting Hickey's quote as research evidence.

Snapshot vs movie

A snapshot measures the code at one instant (Sonar, p.7); a movie mines the whole evolution plus the people dimension (CodeScene, p.9–11). Discriminator: *the snapshot answers "how much?", the movie answers "where to start?"*.

Violations vs technical debt (on the Sonar dashboard)

The dashboard's **10,072 violations** is a rule-breach count; its **Technical Debt 11.0% / \$ 341,563 / 683 man days** is a derived repayment estimate attributed to Duplication, Violations, Complexity, and Comments (BeyondTechnicalDebt p.7). Discriminator: *violations are findings; the debt figure is a costed aggregate over several finding families — and neither is prioritised*.

Code Health vs cyclomatic complexity

Code Health is CodeScene's per-**file** quality band (e.g. `checker.ts` = 1, the worst; the DevTools backend = amber) (p.15, p.17); cyclomatic complexity is a per-**function** path-count metric in the X-Ray table (e.g. 48 for `updateFiberRecursively`) (p.18). Discriminator: *Code Health grades a file; cyclomatic complexity counts a function's branches*.

Change frequency vs recency

The Graves model needs both: total change count predicts faults better than length, but the best model weights **large, recent** changes most heavily (BeyondTechnicalDebt p.14). Discriminator: *a historically-churned but now-quiet file has falling predicted fault rate; a recently-churned file is the live risk*.

Knowledge Loss vs Current Loss vs Simulated Loss vs Off-Boarding Risk

Knowledge Loss is the dashboard field reporting the *current* fraction of abandoned code (e.g. 0% on `checker.ts`, 6% on the DevTools backend) (p.15, p.17). On the simulation map's legend, **Current Loss** = code already abandoned today; **Simulated Loss** = code that would become abandoned in the modelled departure; **Off-Boarding Risk** = code at risk if the selected person leaves; plus **Knowledge** (healthy) and **Inconclusive** (p.22–23). Discriminator: *Current Loss is a fact; Simulated Loss is a what-if; Off-Boarding Risk is the warning band of the what-if*.

Legacy code vs old code

The deck's definition has no age axis: legacy = lacks quality (a *relative* judgement) and/or code *we didn't write ourselves* (p.21). Discriminator: *a brand-new file is legacy the day its only author leaves; a decades-old file with present, knowledgeable owners is not*.

"The technical debt that wasn't" vs ordinary debt

Ordinary debt is substandard code; "the technical debt that wasn't" is *unowned* code — Product #3 with the "?" — whose risk is organisational even if the code is fine (p.21). Discriminator: *one is a defect of the code, the other a defect of the ownership map*.

Change coupling vs static coupling

Static coupling is a structural relationship (imports/calls); change coupling is a *temporal* one (co-change in commits) that can exist with **no** structural link at all (p.19). Discriminator: *static coupling is what the code*

says; *change coupling is what the history proves*. Trap: assuming an empty dependency edge means a change can't propagate.

Hotspot file vs hotspot function

The same logic applies at two granularities: files are ranked codebase-wide (p.12–13); X-Ray then ranks *functions within one file* by the same change-frequency × complexity product (p.16–18). Discriminator: *hotspot analysis finds the file; X-Ray finds the two functions inside it worth fixing*.

Most-changed vs most-dangerous (in the X-Ray table)

The table sorts by change frequency, putting `attach (top-level)` (103 changes, CC 9) on top — but the *targets* are the red-complexity rows `updateFiberRecursively` (CC 48) and `inspectElementRaw` (CC 45) at 24 changes each (p.18). Discriminator: *sort order is not priority order; priority = frequency read against complexity*.

Data supports vs data decides

The conclusion's third and fourth theses: rely on human expertise; support the developer's judgment with data for highest ROI (p.25). Discriminator: *the tool ranks; the human decides*. Trap: any answer implying "refactor whatever CodeScene flags" misses the deck's own bottom line.

Exam Focus

Most likely to be tested:

- **Define technical debt** with the deck's metaphor and the **principal (complexity) vs interest (change frequency)** mapping; explain why "in the way" is the key clause (p.2, p.12).
- **Define a hotspot** as **complexity × change frequency** ("complicated code you work with often"), and explain *why* it is the right place to start repaying debt — i.e. highest interest (p.12).
- **Snapshot vs movie:** contrast conventional static analysis (a point-in-time violation report that can't say where to start, p.6–8) with behavioural/evolutionary analysis (CodeScene — adds **time + organization & people**, p.9–11). Know the pipeline: source + git + JIRA → metrics → ML/pattern detectors → priorities (p.11).
- **Lehman's two laws** (Continuing Change, Increasing Complexity) and how they make debt structurally inevitable (p.3).
- **Empirical justification:** change-history measures predict faults better than size; recent churn matters most (Graves et al., p.14) — this is *why* change frequency is the interest-rate axis.
- **The social dimension:** legacy code = lacks quality (relative) and/or not written by us; knowledge loss, off-boarding risk, "the technical debt that wasn't" (p.20–23). Be ready to explain "*why technical debt isn't just technical*."
- **The conclusion's four points** (p.25): debt is real & language-independent; the VCS holds huge useful information; rely on **human expertise**; use data to **support judgement for highest ROI**.

Likely short-answer / discussion prompts:

- "Your SonarQube report says 11% technical debt / 683 man-days. Why is this not enough to plan repayment, and what would you do instead?" → snapshot critique (precise but not actionable, no "where to start") + hotspot prioritisation (p.7–8, p.12).

- "What is a hotspot and how is it visualised?" → complexity × frequency; nested circles, size = complexity, red = frequency; the priority is the small dark-red circle, not the big pale one (p.12–13).
- "How does X-Ray refine a hotspot analysis?" → drills file → functions, ranks by change-freq × cyclomatic complexity, shows change coupling; makes the refactoring target precise (two functions, not the file) (p.16–19).
- "Explain why technical debt is also a business problem." → long lead times, unpredictability (business) + bugs (product) (p.5); Hickey: complexity kills velocity, every sprint accomplishes less (p.4).

Connect-to-process answers (high-value):

- Position behavioural analysis as the **prioritisation layer over Prefactoring/Postfactoring**: it decides *which* debt to repay so refactoring effort lands where the next **Actualization** is most expensive.
- Use **change coupling** as a behavioural input to **Impact Analysis** (catches hidden impact-set members a static dependency graph misses).
- Note that hotspots are where **Verification** must be most rigorous, since change-history predicts faults concentrate there.
- Distinguish this lecture from LO4: **LO4 = how to refactor** (the repayment mechanism); **L11 = which debt to repay first and when** (the economics/prioritisation).

Definitions to memorise verbatim-ish: technical debt ("...in the way of the stuff that is supposed to be there"); hotspot ("complicated code that you have to work with often"); the two Lehman laws; the principal/interest = complexity/change-frequency mapping.

Self-Test Question Bank

Closed-book questions with model answers, ordered roughly by deck sequence. Cover the answer column and recite.

Definitions and foundations

Q1. Give the deck's working definition of technical debt and its source. A: "*Stuff that isn't supposed to be there and is in the way of the stuff that is supposed to be there*" — *Building Evolutionary Architectures* (p. 110), Neal Ford, Rebecca Parsons, Patrick Kua (BeyondTechnicalDebt p.2). Key clause: "and is in the way" — clutter that obstructs nothing is not (costly) debt.

Q2. State Lehman's two laws as the deck quotes them, and the trap they jointly create. A: Continuing Change — "a system must be continually adapted or it becomes progressively less satisfactory"; Increasing Complexity — "as a system evolves, its complexity increases unless work is done to maintain or reduce it" (p.3). Jointly: you *must* keep changing the system, and every change *tends* to complicate it — so debt accrues unless actively countered.

Q3. Reproduce the core of Hickey's warning and name its source talk. A: If you ignore complexity you will invariably slow down over the long haul; the complexity will eventually kill you "in a way that will make every sprint accomplish less" — Rich Hickey, *Simple Made Easy* (p.4), illustrated by the Easy-vs-Simple speed-over-time chart.

Q4. What two cost categories does the impact slide assign to the business, and what one to the product? A: Business/Roadmap: **long lead times** and **lack of predictability** ("what the business see"); Product: **bugs** ("what the users experience") — both fed by the central "BILL" paying Technical Debt to each side (p.5).

The critique of conventional tools

Q5. The Sonar dashboard reports Technical Debt 11.0% / \$ 341,563 / 683 man days for Apache Tomcat. Why does the deck title this slide "Actionable?" A: Because every figure is precise yet none is prioritised: 10,072 violations (8,794 Major) with no ordering by what the team actually works on; static analysis "will never be able to tell you if that excess code complexity actually matters" (p.7, p.10). The snapshot answers "how much", never "where to start" (p.8).

Q6. Name the four axes of the SIG Maintainability Model spider chart on the Sonar dashboard. A: (A)nalsability, (C)hangeability, (S)tability, (T)estability (p.7).

Q7. What question does the Tower-of-Babel slide pose, and what concept later answers it? A: "Where do you start when you want to pay it back?" (p.8); answered by the **hotspot** — prioritise complexity × change frequency (p.12).

The behavioural turn

Q8. What two dimensions does CodeScene add over traditional tools, per slide 10? A: + **Time aspect** and + **Organization & people**; CodeScene "identifies and prioritizes technical debt based on how the organization works with the code" (p.10).

Q9. Sketch the CodeScene pipeline (inputs → layers → outputs). A: Source Code + Version-Control Data (git) + Project Management Tools (e.g. JIRA) → Code, Process, and Evolutionary Metrics → Pattern Detectors, Machine Learning and Intelligence → Visualizations, priorities, and predictive analytics (p.11).

Hotspots

Q10. Define a hotspot verbatim and map the financial terms onto its two axes. A: "A hotspot is a complicated code that you have to work with often" (p.12). Principal = code complexity; Interest Rate = code change frequency; the hotspot is where both peak.

Q11. In a CodeScene enclosure map, what do circle size and circle colour encode, and which circle is the priority? A: Size = complexity/size (principal); red intensity = change frequency (interest). Priority = the *large-and-dark-red* circles — not merely the biggest (p.12–13).

Q12. Which React packages visibly host the hotspots on the p.13 map? A: `react-dom`, `react-reconciler`, and `react-devtools-shared` (p.13).

Q13. State the Graves et al. findings the deck quotes, with the publication venue. A: IEEE TSE 26(7):653–661, Aug 2000 (Graves, Karr, Marron, Siy): process measures from change history out-predict product metrics; number of changes beats length as a fault predictor; a year-older module has roughly one-third fewer faults; best model = weighted sum of all past changes with large, recent changes weighted most (p.14).

Trends and X-Ray

Q14. Recite the `checker.ts` dashboard from memory. A: TypeScript `src/compiler/checker.ts`: 25,774 LOC; Code Health 1; 785 commits; Main Author Anders Hejlsberg (26%); Knowledge Loss 0%; Defects 1,051 (133% bug fixes); complexity trend rising to ~130,000 (Oct 2015–Apr 2019) over a flat LOC line (p.15).

Q15. What exactly is the "worrisome" signature in a complexity trend? A: Complexity rising while lines of code stay flat — the file is getting harder *per line* (concentrating debt), not merely growing (p.15).

Q16. Why X-Ray at all, in the deck's own words and numbers? A: "We've identified a problematic file but it's still huge!" — the target file is 2,444 LOC (and `checker.ts` is 25,774); X-Ray finds "the most problematic functions" so the fix is scoped to function level (p.15, p.17).

Q17. From the X-Ray table: which function changed most, which two are the real targets, and why the difference? A: Most changed: `attach` (top-level context) — 103 changes but cyclomatic complexity only 9. Targets: `attach.updateFiberRecursively` (CC 48, 220 LOC, 24 changes) and `attach.inspectElementRaw` (CC 45, 180 LOC, 24 changes) — red-flagged complexity with non-trivial churn beats raw churn on tiny principal (p.18).

Q18. Name the X-Ray table's four data columns and its two tabs. A: Columns: Change Frequency, Lines of Code, Cyclomatic Complexity, Overloaded Functions?. Tabs: Structural Recommendations; Change Frequency Distribution (p.18).

Change coupling

Q19. Define change coupling and give the deck's concrete example pair. A: Entities that tend to change together in the same commits regardless of structural references — e.g. `renderer.js` `attach` (top-level context) ↔ `CommitTreeBuilder.js` `updateTree.switch` on the chord diagram (p.19). Uses: hidden impact-set members; Shotgun Surgery detection (p.19, p.24).

Legacy and off-boarding

Q20. Give the deck's two-bullet characterisation of legacy code and explain "The Technical Debt That Wasn't". A: Legacy code: lacks in quality (relative perspective); **we didn't write ourselves** (p.21). "The Technical Debt That Wasn't": of three products, two have owners (thumbs-up) and one has only "?" — abandoned code whose liability is purely organisational (p.21).

Q21. List the five legend states of the off-boarding simulation and the workflow demonstrated. A: Knowledge, Current Loss, Simulated Loss, Off-Boarding Risk, Inconclusive. Workflow: scope to Directory `react`, filter developers ("brian"), check Brian Vaughn, re-render — blue map "before" turns red-patched "after they leave ..." (p.22–23).

Q22. Why is Brian Vaughn the perfect choice of simulation subject within the deck's own narrative? A: He is the Main Author (80%) of the `react-devtools-shared/src/backend` hotspot from p.17 — the simulation shows the repository-wide knowledge crater his departure would cause, unifying the technical and social halves of the lecture (p.17, p.22–23).

Breadth and conclusions

Q23. Reproduce the "There's much more in CodeScene" list with its nesting. A: Change coupling; Microservices (Shotgun surgery, Team conflicts, Technical sprawl); Proactive warnings; Retrospectives; Delivery Performance; Branch Analyses (p.24).

Q24. State the four concluding theses. A: (1) Technical debt is a real problem regardless of programming language; (2) there's a huge amount of useful information stored in your version control system; (3) ultimately, you need to rely on human expertise; (4) support your developer's judgment and experience with data to get the highest ROI (p.25).

Q25. Integrative: a change request's impact set includes a file that is a hotspot with high change coupling to two other files and an 80% single author. Write the risk assessment. A: Expect expensive Actualization (hotspot = complex + churned, p.12) and elevated regression risk (change

history predicts faults, p.14) → budget Prefactoring on its worst functions via X-Ray (p.16–18) and intensify Verification; add the two coupled files to the impact set (p.19); before touching it, mitigate the knowledge concentration (pair/document) since an 80% single-author file is one departure away from legacy (p.17, p.20–23). Let the data inform, and the engineer decide (p.25). (*Process mapping drawn by this guide.*)

Source Map

Slide(s)	Title / Content	What to extract
p.1	"Beyond Technical Debt" title (tortoise on a road)	Theme: debt slows you down for the long haul; the lecture goes <i>beyond</i> the naive snapshot view.
p.2	"Technical Debt" definition	Working definition (Ford/Parsons/Kua, <i>Building Evolutionary Architectures</i> p.110): "in the way of the stuff that is supposed to be there."
p.3	"Lehman" — Laws of Software Evolution	Continuing Change; Increasing Complexity → debt is structurally inevitable.
p.4	"Complexity Kills Development Speed"	Rich Hickey (<i>Simple Made Easy</i>): ignored complexity slows you over the long haul, "kills" velocity; speed-vs-time chart (Easy vs Simple).
p.5	"Technical Debt Has Impact on Business & Product"	Business: long lead times, lack of predictability. Product: bugs users experience. "BILL" paid to both sides.
p.6	"Conventional Tools — What is missing?"	Static tools give a snapshot; setup for the critique.
p.7	"Actionable?" (Sonar dashboard)	162,306 LOC; 10,072 violations (8,794 Major); Technical Debt 11.0% / €341,563 / 683 man days. Precise but not actionable — no "where to start."
p.8	"Thousands of Years of Technical Debt" (Tower of Babel)	"Where do you start when you want to pay it back?" — prioritisation problem.
p.9	"Codescene" — "movie rather than a snapshot"	The methodological turn: evolutionary, not point-in-time.
p.10	"Codescene" — Time + Organization & people	Static analysis can't tell you if complexity <i>matters</i> ; CodeScene prioritises based on how the organization works with the code . +Time aspect, +Organization & people.
p.11	CodeScene pipeline diagram	Inputs: source + git + JIRA → Code/Process/Evolutionary Metrics → Pattern Detectors/ML → Visualizations, priorities, predictive analytics.
p.12	"Hotspot" definition	"Complicated code you work with often" = Principal (complexity) × Interest Rate (change frequency) .
p.13	ReactJS hotspot map	Nested-circle enclosure diagram; size = complexity, red = change frequency; hotspots in react-reconciler, react-devtools-shared.
p.14	"Hotspots Are Also Common Sources of Bugs"	Graves et al. (2000): change-history measures predict faults better than size; #changes > length; recent churn weighted most.
p.15	"Worrisome Trends" (TypeScript checker.ts)	25,774 LOC; 785 commits; 1,051 defects (133% bug fixes); Code Health 1; complexity curve rising far faster than flat LOC.
p.16	"X-Ray — Deep Dive" (title)	Drill from file to function.
p.17	"Problematic file but still huge"	Rationale for X-Ray: find the worst functions in a big hotspot.
p.18	X-Ray function table (React attach.*)	Functions ranked by change freq × cyclomatic complexity; worst: updateFiberRecursively (48), inspectElementRaw (45).

Slide(s)	Title / Content	What to extract
p.19	"Change Coupling" (chord diagram)	Files/functions that change together; hidden temporal dependency; Shotgun Surgery; feeds impact analysis.
p.20	"Legacy Code — Why technical debt isn't just technical" (title)	The social dimension.
p.21	"Legacy Code" + "The Technical Debt That Wasn't"	Legacy = lacks quality (relative) + not written by us; abandoned/unowned code (Product #3 = "?").
p.22	"How quickly can you turn your codebase into legacy code?" (before)	Off-boarding simulation; legend: Knowledge/Current Loss/Simulated Loss/Off-Boarding Risk/Inconclusive; pick developers.
p.23	"After they leave ..." (after)	Map turns red — code that becomes legacy/off-boarding risk if author departs; bus-factor made predictive.
p.24	"There's Much More in CodeScene"	Change coupling, microservices (shotgun surgery, team conflicts, technical sprawl), proactive warnings, retrospectives, delivery performance, branch analyses.
p.25	"To Conclude..."	Debt is real & language-independent; VCS holds huge useful info; rely on human expertise ; support developers' judgement with data for highest ROI .
p.26	"Resources"	adamtornhill.com; tools at adamtornhill.com/code/crimescenetools.htm (Adam Tornhill / <i>Your Code as a Crime Scene</i>).

Fine print from the page renders (verification notes)

A second full visual pass over all 26 page renders (for this expansion) confirms the Source Map above and adds the following points of precision, recorded so the corpus carries the slide-accurate values:

- **p.2 author spelling.** The slide credits "Neal Ford, Rebecca Parsons, and Patrick **Kia**" — the book's actual co-author is Patrick **Kua**; treat "Kia" as a slide typo and write "Kua" in answers (BeyondTechnicalDebt p.2).
- **p.7 currency symbol.** The Technical Debt panel prints the monetary value with a **dollar sign** — "\$ **341,563**" — alongside 11.0% and 683 man days. The analysed project is identified at the dashboard's foot: **Key: org.apache.tomcat, Language: java** (BeyondTechnicalDebt p.7).
- **p.7 severity detail.** The violations breakdown reads Blocker 0, Critical 0, **Major 8,794** (circled), Minor 65, Info 1,213 — note the absence of any Blocker/Critical items despite the 683-man-day debt estimate, reinforcing that severity labels alone don't capture cost (BeyondTechnicalDebt p.7).
- **p.15 truncated field.** The last dashboard row ("Last ...", i.e. last-modified) is partially cut off at the slide edge; the visible value reads "0 months ago" (BeyondTechnicalDebt p.15).
- **p.15 trend axes.** The Complexity Trend's y-axis is labelled "Complexity (ws)" scaled to ~130,000, the x-axis runs October 2015 → April 2019, and three series are plotted: Complexity (red), Lines of Code (blue), Code Comments (green); caption: "Click on a point to diff the code changes" (BeyondTechnicalDebt p.15).
- **p.18 full table.** The X-Ray table contains **nine visible rows**, including two often-omitted ones — `attach.flushInitialOperations` (29 / 51 / 6) and `attach.recordUnmount` (18 / 50 / 9) — plus the **"Overloaded Functions?"** column (value 1 on every visible row) and per-row trend/view-code buttons (BeyondTechnicalDebt p.18).
- **p.22–23 instruction text.** Both simulation slides repeat the instruction *"Simulate the effects of a planned off-boarding if some developers leave your organization"*; the before-slide's developer filter shows the typed string "brian" resolving to the checkbox "Brian Vaughn", and the after-slide lists "Simulated offboarded authors: Brian Vaughn" (BeyondTechnicalDebt p.22–23).

- **Defect-percentage semantics.** The "(133 % Bug Fixes)" annotation on `checker.ts`'s 1,051 defects (and "(20 % Bug Fixes)" on the DevTools backend's 24) is printed without further definition on the slides; the safe reading is that it expresses defect-linked change activity relative to the file's other change activity, with >100% meaning bug-fixing dominates — the slides themselves do not spell the formula out (BeyondTechnicalDebt p.15, p.17).